



POLITECNICO DI BARI

Dipartimento di Meccanica, Matematica e Management

CORSO DI LAUREA IN INGEGNERIA MECCANICA

Tesi di Laurea in PRINCIPI DI INGEGNERIA ELETTRICA

**TSP E ISING MACHINES SPINTRONICHE: ANALISI
METODOLOGICA PER PROBLEMI NP-HARD E
CRITICA DEL COMPROMESSO PRESTAZIONALE**

Relatore:

Chiar.mo Prof. Vito Puliafito

Correlatore:

Dott. Ing. Angelantonio Squicciarini

Laureando:

Antonio Papagni

Anno Accademico 2024–2025



Politecnico
di Bari

**LIBERATORIA ALLA CONSULTAZIONE DELLA TESI DI LAUREA DI CUI
ALL'ART.4 DEL REGOLAMENTO DI ATENEIO PER LA CONSULTAZIONE
DELLE TESI DI LAUREA (D.R. n. 479 del 14/11/2016).**

Il sottoscritto Antonio Papagni matricola 588433

Corso di Laurea Ingegneria Meccanica

autore della presente tesi di Laurea dal titolo:

TSP E ISING MACHINES SPINTRONICHE: ANALISI METODOLOGICA PER
PROBLEMI NP-HARD E CRITICA DEL COMPROMESSO PRESTAZIONALE

Parole chiave: TSP, ISING MACHINE, NP-HARD, TRADE-OFF

Abstract: LA TESI ANALIZZA L'APPLICAZIONE DELLE PIM BASATE SU
SMTJ PER LA RISOLUZIONE EFFICIENTE DEL TSP, EVIDENZIANDO
COME L'INTEGRAZIONE TRA RUMORE TERMICO E ANNEALING MIGLIORI
IL TRADE-OFF TRA ACCURATEZZA E TEMPO DI CALCOLO

☒ Autorizza

☐ Non autorizza

la consultazione della presente tesi, fatto divieto a chiunque di riprodurre in tutto o in
parte quanto in essa contenuto.

Bari, 25 Ottobre 2025

Firma

Antonio Papagni

Indice

Introduzione	1
Capitolo 1:	3
Contesto, Obiettivi e Inquadramento Metodologico.....	3
1.1 Il Contesto del Calcolo Non Convenzionale: Von Neumann Bottleneck e la crescente domanda di efficienza energetica	3
1.1.1 Il Problema P vs NP e la Sfida Computazionale	4
1.1.2 Riflessione Sociale e Rilevanza: perché risolvere i problemi NP-Hard.....	5
1.2 Il Problema TSP e la Mappatura Universale: Introduzione al Traveling Salesman Problem (TSP) come paradigma di ottimizzazione ..	7
1.3 Focus Critico: analisi comparativa, valutazione delle SMTJ e studio del Trade-off Prestazionale.....	8
Capitolo 2:	11
Fondamenti Teorici e Paradigmi di Soluzione.....	11
2.1 L'Hamiltoniana di Ising (QUBO) e Mappatura Rigorosa: Origine fisica e definizione dell'Hamiltoniana	11
2.1.1 Minimo Energetico vs Soluzione Algoritmica: Perché il "quasi Ground State" non garantisce una soluzione quasi-ottimale	13
2.2 Paradigmi di Ising Machine a Confronto	15
2.2.1 Ising Machines Coerenti (CIM)	16
2.2.2 Ising Machines Probabilistiche (pIM)	16
2.2.3 Analisi Critica: Prestazioni a Confronto	17
2.3 La Piattaforma Spintronica SMTJ: Il p-bit come spin stocastico implementato con SMTJ e l'origine del rumore intrinseco	18
2.4 Architetture per la Velocità: Concetto di Gibbs Sampling (sincrono, pseudo-asincrono) e Sparsificazione per il parallelismo	19
2.4.1 Architetture per la Scalabilità (Cross-bar Array e Sparsificazione).....	20
Capitolo 3:	21
Il TSP e la Critica dei Metodi Algoritmici Tradizionali.....	21

3.1 Rassegna delle Soluzioni Classiche al TSP: Algoritmi Esatti (Branch and Bound) e Metaeuristici (Genetic Algorithm)	21
3.1.1 Metodi Algoritmici Esatti (Branch and Bound)	21
3.1.2 Metaeuristiche e Algoritmi Ispirati alla Natura (Genetic Algorithm)	22
3.2 Analisi Critica delle Strategie Greedy: Vantaggi di velocità e Svantaggi di qualità	23
3.2.1 Rischio di Convergenza Prematura e l'Incapacità delle Euristiche di Fuggire dai Minimi Locali	23
3.3 Simulated Annealing (SA) Algoritmico: Il Precursore Stocastico	24
3.3.1 Dipendenza dalla Curva di Raffreddamento e Parametri Critici	25
Capitolo 4:	28
Implementazione Critica: TSP, Parametri e Trade-off Prestazionale	28
4.1 Mappatura del TSP sull'Hamiltoniana: Formulazione dei Termini di Vincolo e Costo	29
4.1.1 Definizione delle Variabili di Spin	29
4.1.2 Formulazione dei Termini di Vincolo ($H_{vincoli}$)	30
4.1.3 Formulazione del Termine di Costo (H_{costo})	31
4.1.4 L'Hamiltoniana Finale e la Calibrazione (TSP vs CTSP)	31
4.2 Strategie Dinamiche e Ruolo del Rumore: L'Annealing come guida nell'esplorazione del paesaggio energetico	33
4.2.1 Il Ruolo Centrale del Rumore Stocastico	33
4.2.2 Schemi Dinamici di Annealing a Confronto	34
4.3 Analisi dei Parametri Critici: Studio di sensibilità sull'impatto di variazioni nei Parametri Critici sulla soluzione IM	37
4.3.1 Parametri di Mappatura e Penalità	37
4.3.2 Parametri di Annealing e Dinamica Stocastica	38
4.3.3 Sensibilità alle Variazioni Fisiche (Noise Sources)	38
4.4 Studio del Trade-off Prestazionale: L'analisi del Compromesso tra la "Time to Solution (TTS)" e l'Accuratezza della Soluzione	39
4.4.1 Definizione e Metriche del Trade-off	39
4.4.2 L'Impatto Critico dello Schema di Annealing (TTS vs Iterazioni)	40
4.4.3 Metriche Fisiche: Energia e Velocità	41
4.4.4 Il Trade-off nel Design dell'Hamiltoniana	42

Scalabilità e Risultati	43
5.1 Il Paradigma TSP-Ising applicato al Routing Sanitario: Obiettivi, Giustificazione Metodologica e Risultati nell'Home Health Care Problem.....	43
5.1.1 Obiettivi dell'Ottimizzazione nel Contesto Sanitario.....	44
5.1.2 Giustificazione Metodologica: Il Vantaggio del Calcolo Spintronico	45
5.1.3 Risultati e Impatto dell'Ottimizzazione Sanitaria	46
5.2 Analisi Dettagliata delle Simulazioni TSP Tramite Modello SHIL.....	46
5.2.1 Metodologia Applicata	47
5.3 Analisi Dettagliata dei Risultati (SHIL Puro) e Variazione Parametrica.....	48
5.3.1 Caso N=6 Città	48
5.3.2 Caso N=10 Città	51
5.3.3 Caso N=12 Città: Fallimento di Convergenza (SHIL Puro)	55
5.4 Analisi Dettagliata dei Risultati (SHIL Ibrido)	58
5.5 Conclusioni Generali: Sintesi Qualità/Velocità.....	62
Capitolo 6:	64
Conclusioni e Prospettive Future: Vantaggio competitivo finale delle SMTJ-pIM sull'ottimizzazione e direzioni future della ricerca	64
6.1 Sintesi del Vantaggio Competitivo SMTJ-pIM.....	64
6.2 Direzioni Future della Ricerca.....	65
Bibliografia	67
Appendice A:	69
Codice <i>MATLAB</i> approccio SHIL Ibrido con Two Opt (N = 12):.....	69
Appendice B:	74
Codice <i>MATLAB</i> approccio SHIL senza <i>Two Opt</i> (N = 12):	74
Appendice C:	79
Codice <i>MATLAB</i> approccio <i>Brute Force</i> (N=12):	79

Introduzione

Negli ultimi anni, l'incremento della complessità computazionale dei problemi di ottimizzazione combinatoria ha posto un limite notevole ai metodi algoritmici tradizionali. Tra questi, il *Traveling Salesman Problem* (TSP) rappresenta uno dei casi principali: la sua natura **NP-Hard** rende la risoluzione esatta limitata tramite approcci deterministici, in quanto il numero di configurazioni cresce in modo fattoriale con il numero delle città che si va a considerare. Nonostante lo sviluppo di algoritmi esatti e metaeuristici — come *Branch and Bound*, *Genetic Algorithms* o *Simulated Annealing* — il *trade-off* (compromesso) tra **tempo di calcolo** (*Time to Solution*) e **qualità della soluzione** rimane un punto critico.

In tale contesto, le **Ising Machines (IM)** si configurano come una promettente alternativa di calcolo non convenzionale, in grado di “tradurre” problemi di ottimizzazione in termini di **minimizzazione energetica**. L'approccio analizzato si basa sul formalismo del **modello di Ising**, dove ogni configurazione del sistema è rappresentata da uno stato energetico, e la soluzione ottimale corrisponde all'individuazione del *Ground State*. Per consentire la mappatura dei vincoli e delle funzioni obiettivo in un formato compatibile con la funzione Hamiltoniana si sfrutta la formulazione **Quadratic Unconstrained Binary Optimization (QUBO)**, che verrà trattata nei capitoli seguenti.

Tra le diverse implementazioni studiate, le **Probabilistic Ising Machines (pIM)** basate su **Superparamagnetic Tunnel Junctions (SMTJ)** rappresentano quella più avanzata e innovativa. Questi dispositivi spintronici sfruttano il **rumore termico ambientale** come risorsa computazionale, introducendo una stocasticità (casualità) fisicamente intrinseca che simula il comportamento di un *Simulated Annealing hardware*. Codesto approccio riduce i consumi energetici e permette una **ricerca parallela** delle soluzioni, superando il collo di bottiglia architetturale del modello di *von Neumann*.

L'obiettivo di questa tesi è duplice: da un lato, formalizzare il processo di **mappatura del TSP nel modello QUBO**, analizzando in modo rigoroso la

costruzione dell'Hamiltoniana e la calibrazione dei coefficienti di penalità; dall'altro, valutare le prestazioni di un'architettura **SMTJ-pIM** attraverso simulazioni numeriche in ambiente *MATLAB*. L'analisi si concentra maggiormente sul ***trade-off* prestazionale** tra accuratezza della soluzione e tempo di convergenza, individuando le condizioni ottimali per massimizzare l'efficienza del sistema.

L'intero studio ha come obiettivo principale la dimostrazione che l'integrazione tra **rumore fisico intrinseco** e ***annealing* controllato**, consente alle pIM di ottenere un vantaggio concreto nella risoluzione di problemi combinatori complessi, aprendo prospettive significative per l'evoluzione dei paradigmi di calcolo non convenzionali.

Capitolo 1:

Contesto, Obiettivi e Inquadramento Metodologico

La crisi di efficienza energetica e le sfide di scalabilità che affliggono l'informatica moderna, evidenziate dal ***Von Neumann Bottleneck***, hanno reso obsoleti i metodi convenzionali per la risoluzione dei problemi di ottimizzazione combinatoria ***NP-Hard***.

Questo primo capitolo analizza il fallimento strutturale delle architetture classiche nel gestire la complessità esponenziale del ***Traveling Salesman Problem (TSP)***, che viene adottato come paradigma di ottimizzazione universale. Nella sezione conclusiva saranno anche definiti gli obiettivi critici della ricerca: l'analisi comparativa dei paradigmi ***Ising Machine (IM)***, la valutazione dell'implementazione ***spintronica (SMTJ)*** come soluzione energeticamente efficiente, e lo studio del ***Trade-off Prestazionale*** tra la qualità della soluzione e il tempo di calcolo richiesto (TTS).

1.1 Il Contesto del Calcolo Non Convenzionale: Von Neumann Bottleneck e la crescente domanda di efficienza energetica

L'attuale panorama computazionale è caratterizzato da una domanda esponenziale di capacità di calcolo, guidata soprattutto dall'espansione dei carichi di lavoro legati all'intelligenza artificiale e ai problemi di ottimizzazione combinatoria su larga scala. Nonostante i progressi nella miniaturizzazione dei *transistor* (legge di *Moore*), il modello di calcolo predominante, ossia l'architettura di *Von Neumann*, ha raggiunto limiti di efficienza energetica e prestazionale insostenibili.

Tale "architettura di *Von Neumann*", fondata sulla separazione fisica e logica tra l'unità centrale di elaborazione (CPU) e la memoria, è afflitta da un limite strutturale noto come ***Von Neumann Bottleneck*** [1]. Questo "collo di bottiglia" si origina dal limitato *throughput* del mezzo di comunicazione, costringendo la CPU ad attendere il continuo trasferimento dei dati (*data shuttling*) dalla

memoria. Il risultato è un elevato consumo energetico e una latenza sistemica, poiché la potenza spesa per lo spostamento dei dati supera di gran lunga quella dedicata al calcolo effettivo. Tale inefficienza ha reso impraticabile l'approccio convenzionale per la risoluzione dei problemi che richiedono un'elevata intensità di calcolo e campionamento, spingendo la ricerca verso l'esplorazione di paradigmi di **calcolo non convenzionale** (*Unconventional Computing* o *Beyond-CMOS*).

L'obiettivo primario di queste nuove architetture è duplice: superare il *Von Neumann Bottleneck* mediante l'integrazione di memoria ed elaborazione (*in-memory computing*) e sfruttare fenomeni fisici intrinseci per ottenere un'efficienza energetica di gran lunga superiore. In questo contesto, le **Ising Machines**, che mappano la computazione sulla dinamica di sistemi fisici per la ricerca di stati energetici minimi, si presentano come la soluzione architetutturale più promettente per affrontare la crisi di efficienza e la crescente domanda energetica del settore tecnologico.

1.1.1 Il Problema P vs NP e la Sfida Computazionale

L'insorgenza della crisi di efficienza energetica, dovuta al collo di bottiglia di *Von Neumann*, è aggravata dalla natura complessa dei problemi che la società moderna richiede di risolvere: i cosiddetti “**problemi NP-Hard**”. La sfida computazionale si concentra sul divario teorico tra la classe di problemi **P** e la classe **NP**.

- **Classe P (Tempo Polinomiale):** comprende i problemi per i quali la soluzione può essere trovata da un algoritmo deterministico in un tempo limitato da una funzione polinomiale della dimensione dell'*input* di partenza. Tali problemi sono considerati trattabili;
- **Classe NP (Tempo Polinomiale Non Deterministico):** riguarda i problemi per i quali, pur non essendo nota una soluzione efficiente per la ricerca, la verifica di una soluzione candidata può essere completata in tempo polinomiale.

Il dibattito scientifico sul fatto che $P = NP$ (ovvero, se i problemi la cui soluzione è facile da verificare siano anche facili da trovare) rimane irrisolto. Tuttavia, il consenso attuale assume che $P \neq NP$, il che implica che i problemi più complessi non ammettono soluzioni algoritmiche efficienti la maggior parte delle volte.

I problemi **NP-Hard** (non deterministici) costituiscono la vera sfida per il calcolo convenzionale. Un problema di ottimizzazione è definito *NP-Hard* se risulta "tanto difficile" quanto il più difficile dei problemi NP. La complessità di calcolo per trovare la soluzione ottima o quasi-ottima cresce tipicamente in modo **esponenziale** rispetto alla dimensione del problema. Questo comporta un **fallimento strutturale** per le architetture classiche: un modesto aumento della dimensione del problema può richiedere un tempo di calcolo che **supera la vita utile dell'intero universo**, rendendo i problemi rapidamente intrattabili per istanze reali e di grandi dimensioni.

Tra i problemi *NP-Hard* di maggiore rilevanza per l'informatica non convenzionale, si annoverano:

- Il **Traveling Salesman Problem (TSP)**;
- I problemi di **Massima Soddisfacibilità (Max-SAT)**;
- I problemi di **Massimo Taglio (Max-Cut)**;
- I problemi di **Ottimizzazione di Scheduling**.

La necessità di superare questa **sfida computazionale esponenziale** è la principale motivazione dietro l'adozione di paradigmi di calcolo alternativi, in particolare le **Ising Machines**, che cercano di aggirare l'esplosione combinatoria mappando la complessità su un sistema fisico che evolve naturalmente verso il minimo energetico.

1.1.2 Riflessione Sociale e Rilevanza: perché risolvere i problemi NP-Hard

La difficoltà teorica dei problemi **NP-Hard**, descritta dal divario P vs NP , non è solo un tema accademico ma si riflette concretamente in molti ambiti applicativi

di tipo scientifico o industriale. La capacità di risolvere rapidamente e con elevata efficienza energetica le istanze di questi problemi è un requisito fondamentale per l'innovazione tecnologica moderna.

Tra i problemi *NP-Hard*, il ***Traveling Salesman Problem (TSP)*** è il caso di studio per eccellenza, tanto da fungere da *benchmark* per l'ottimizzazione combinatoria e per la valutazione di nuove e necessarie architetture di calcolo, come le *Ising Machines*.

Nel caso del TSP, si cerca il percorso più breve che tocchi ogni città una sola volta, per poi tornare al punto di partenza. La sua rilevanza, tuttavia, non si limita alla logistica, ma si estende a problemi con una struttura isomorfa, ovvero che possono essere mappati sul TSP.

Alcuni esempi chiave potrebbero riguardare ambiti come:

- **Logistica e Pianificazione:** ottimizzazione dei percorsi di consegna;
- **Bioinformatica:** ricostruzione di sequenze genomiche e allineamento di DNA;
- **Progettazione Elettronica:** ottimizzazione del percorso dei fili nelle schede a circuiti stampati e pianificazione dei test;
- **Fisica Statistica/Chimica:** configurazioni di energia minima in sistemi con interazioni tra particelle.

Il numero di possibili percorsi distinti (ovvero le soluzioni) in un'istanza del TSP cresce con il **fattoriale** della dimensione del problema (il numero di città N), rendendo l'approccio "*brute force*" impraticabile già per istanze modeste. Questa esplosione dello spazio delle soluzioni (*combinatorial explosion*) motiva il passaggio da algoritmi di ricerca esaustiva a:

1. **Metodi Euristici e Approssimati:** come gli algoritmi "**genetici**" o "**Greedy**", che trovano soluzioni quasi-ottimali (che saranno analizzati più nel dettaglio nel Capitolo 3);

2. **Mappatura su Sistemi Fisici:** sfruttando la dinamica fisica, dove lo stato di energia minima di un sistema rappresenta la soluzione ottima del problema di ottimizzazione [2] .

Per superare i limiti di scalabilità e di efficienza energetica nel trattamento di istanze grandi del TSP, è necessario un modello che superi i concetti di istruzione sequenziale e memoria separata di *Von Neumann*.

Le *Ising Machines* (IM) rappresentano, in questo senso, un possibile ponte concettuale poiché caratterizzati da:

- **Natura Fisica:** mappano il problema *NP-Hard* su una funzione di energia (l'*Hamiltoniana di Ising*);
- **Efficienza Energetica:** sfruttano la dinamica del sistema per evolvere autonomamente verso gli stati di minima energia, riducendo drasticamente il *data shuttling* (spostamento e trasferimento di dati) e, quindi, i consumi energetici; un aspetto cruciale evidenziato nello studio sulle SMTJ-IM per il TSP.

L'adozione di un modello come il Modello di *Ising* è pertanto il passo logico successivo per affrontare la sfida computazionale in modo efficiente e scalabile.

1.2 Il Problema TSP e la Mappatura Universale: Introduzione al Traveling Salesman Problem (TSP) come paradigma di ottimizzazione

L'adozione di un paradigma di **calcolo non convenzionale** è motivata dalla necessità di superare la barriera teorica posta dalla classe di problemi *NP-Hard*, tra cui il *Traveling Salesman Problem*. Esso funge da *benchmark* cruciale per la valutazione di nuove architetture (incluse le *Ising Machines*) e la difficoltà risiede nell'**intrattabilità combinatoria** dello spazio delle soluzioni: il numero di percorsi distinti cresce come il fattoriale della dimensione del problema (N), ovvero $\left(\frac{(N-1)!}{2}\right)$.

Per istanze realistiche, ad esempio con $N=70$ città, il numero di combinazioni supera 10^{98} , rendendo i metodi algoritmici convenzionali inefficaci a causa del **Time to Solution (TTS)** proibitivo. Per superare questa sfida serve un approccio completamente diverso rispetto alla tradizionale logica sequenziale del calcolo deterministico.

L'importanza del TSP risiede nella sua capacità di agire come **paradigma di ottimizzazione universale**. Questo problema può essere rappresentato, in modo rigoroso, sul **Modello di Ising** (o sulla sua formulazione equivalente, **QUBO** - *Quadratic Unconstrained Binary Optimization*). Questa **mappatura** trasforma il problema *NP-Hard* di ricerca del percorso più breve nella ricerca dello **stato fondamentale** (*Ground State*) del sistema energetico.

Il modello di *Ising* fornisce il linguaggio matematico e fisico necessario per l'implementazione su *hardware* non convenzionale:

$$H(\sigma) = - \sum_{i < j} J_{ij} \sigma_i \sigma_j - \sum_i h_i \sigma_i$$

Dove i termini J_{ij} (coefficienti di accoppiamento) e h_i (campi esterni) incapsulano rispettivamente i **vincoli** e i **costi** del problema TSP. La soluzione del problema coincide, in pratica, con la configurazione di *spin* (σ_i) che porta il sistema alla minima energia $H(\sigma)$.

Questa conversione è il passo concettuale fondamentale per sfruttare l'efficienza energetica delle **Ising Machines Spintroniche** per il calcolo.

1.3 Focus Critico: analisi comparativa, valutazione delle SMTJ e studio del Trade-off Prestazionale

Dopo aver definito il contesto di crisi architetturale e la complessità dei problemi *NP-Hard*, la tesi si concentra sull'analisi di un paradigma di calcolo alternativo: le **Probabilistic Ising Machines (pIM)** implementate mediante tecnologia spintronica.

L'obiettivo del lavoro è fornire un'analisi comparativa rigorosa e una valutazione approfondita di questa piattaforma di calcolo **non convenzionale**, strutturata su tre punti principali:

1. Analisi Comparativa: pIM vs. CIM

Il primo focus metodologico è il paragone tra i due principali paradigmi di *Ising Machines* (IM): le **Coherent Ising Machines (CIM)** e le **Probabilistic Ising Machines (pIM)**.

- **Differenza Fondamentale:** le CIM mappano lo *spin* sulla fase di oscillatori coerenti mentre le pIM implementano lo *spin* come un ***p-bit*** (*probabilistic bit*), ovvero un'entità stocastica che fluttua intrinsecamente tra due stati (0 e 1, o ± 1);
- **Superiorità Prospettica:** diversi studi suggeriscono che le **pIM possano superare le CIM** in termini di numero di iterazioni necessarie per convergere a una soluzione quasi-ottimale. Tale vantaggio è attribuibile alla capacità delle pIM di sfruttare la **stocasticità (rumore)** per fuggire efficacemente dai minimi locali, limitazione intrinseca che affligge gli schemi coerenti e deterministici.

2. Valutazione delle SMTJ e il Ruolo del Rumore Intrinseco

Il secondo punto si basa sull'analisi dell'implementazione fisica del *p-bit*. Il lavoro si concentra sull'architettura implementata con **Superparamagnetic Tunnel Junctions (SMTJ)**.

- **Hardware Avanzato:** le SMTJ sono dispositivi **spintronici** compatibili con la tecnologia CMOS, rappresentando la piattaforma hardware più avanzata per realizzare *p-bits* stocastici;
- **Efficienza Energetica Massima:** le SMTJ risultano particolarmente efficienti poiché riescono a sfruttare il **rumore termico ambientale** (k_bT) come fonte naturale e intrinseca di fluttuazioni di *spin*. Questo approccio riduce il consumo energetico a livelli straordinariamente bassi, nell'ordine dei **femtojoule (fJ) per operazione**;

- **Base per l'*annealing*:** la stocasticità intrinseca generata dalle SMTJ è l'elemento principale per implementare i cicli di ***annealing* (ricottura)**, che guidano il sistema verso il minimo energetico (ossia la soluzione del TSP).

3. Studio del *Trade-off* Prestazionale

Il terzo obiettivo critico è l'analisi del ***Trade-off* (Compromesso) Prestazionale** tra la rapidità di calcolo e la qualità della soluzione.

- **Metriche del compromesso:** l'analisi si concentra sulla relazione inversa tra la ***Time to Solution*** (tempo necessario per trovare una soluzione) e l'**accuratezza della soluzione** (misurata dal costo medio, C_{Medio} , o dalla deviazione standard, $Std(C)$);
- **Impatto dei parametri critici:** la tesi valuta come la scelta dei **parametri di mappatura** (coefficienti di penalità e nell'Hamiltoniana) e dei **parametri di *annealing*** (strategia dinamica e rumore) influenzano questo compromesso. Una buona calibrazione dei parametri deve non solo garantire il rispetto dei vincoli del TSP, ma anche massimizzare la probabilità di convergenza al *Ground State* (qualità del percorso), pur mantenendo un TTS basso e competitivo.

L'insieme di questi focus definisce l'inquadramento metodologico della tesi, che passa ad analizzare i fondamenti teorici per l'implementazione pratica del TSP su piattaforme SMTJ-pIM.

Capitolo 2:

Fondamenti Teorici e Paradigmi di Soluzione

Nel primo capitolo si è motivata la necessità di un cambio di paradigma per affrontare i problemi **NP-Hard**: risulta chiaro definire ora il collegamento concettuale tra la sfida informatica e la soluzione fisica.

L'analisi si apre con la formalizzazione del problema tramite l'**Hamiltoniana di Ising** e la formulazione **QUBO**. Una volta esplorate le implicazioni del **minimo energetico** rispetto al comportamento ottimo dell'algoritmo e il problema dei minimi locali, tale capitolo confronta i paradigmi architetturali di calcolo non convenzionale: le **Coherent Ising Machines (CIM)** e le **Probabilistic Ising Machines (pIM)**. Nella sezione conclusiva, il focus si sposta sull'*hardware* chiave della tesi: la **piattaforma spintronica SMTJ**. Si descrive inoltre come il **p-bit stocastico** sia realizzato per sfruttare il rumore termico intrinseco e come le architetture di **Gibbs Sampling** e **sparsificazione** garantiscano velocità e scalabilità per la risoluzione del TSP su problemi di larga scala.

2.1 L'Hamiltoniana di Ising (QUBO) e Mappatura Rigorosa: Origine fisica e definizione dell'Hamiltoniana

Il **modello di Ising**, sviluppato originariamente da *Wilhelm Lenz* e rifinito da *Ernst Ising* nel 1925, è uno dei modelli più influenti della fisica statistica; la sua importanza (come evidenziato nel *paper* scientifico "*The importance of the Ising model*" di *B.M. McCoy* e *J.-M. Maillard*) risiede nell'essere il primo sistema meccanico-statistico che esibisce una **transizione di fase** (ferromagnetismo), servendo da pietra miliare per la comprensione di fenomeni di *scaling* e universalità.

Il modello di *Ising* descrive un sistema di **dipoli magnetici elementari** (*spin*) disposti su un reticolo. Ogni *spin* σ_i può trovarsi in due stati binari discreti ossia

allineato con il campo magnetico applicato ($\sigma_i = +1$, definito “*spin-up*”) o in direzione contraria ($\sigma_i = -1$, definito “*spin-down*”).

La funzione Hamiltoniana governa l'interazione tra gli *spin* e l'ambiente circostante, definendo l'energia totale del sistema [2] :

$$H(\sigma) = - \sum_{i < j} J_{ij} \sigma_i \sigma_j - \sum_i h_i \sigma_i$$

Si definiscono di seguito i due contributi principali al termine energetico:

- **Termine di accoppiamento** ($-\sum_{i < j} J_{ij} \sigma_i \sigma_j$): rappresenta l'interazione tra gli *spin* adiacenti. Nell'analisi ferromagnetica, un coefficiente di accoppiamento positivo ($J_{ij} > 0$) favorisce l'allineamento degli *spin* (consentendo la magnetizzazione), mentre un accoppiamento negativo ($J_{ij} < 0$) favorisce il fenomeno dell'anti-ferromagnetismo;
- **Termine di campo esterno** ($-\sum_i h_i \sigma_i$): rappresenta l'energia che ogni *spin* acquisisce interagendo con un campo magnetico esterno (h_i).

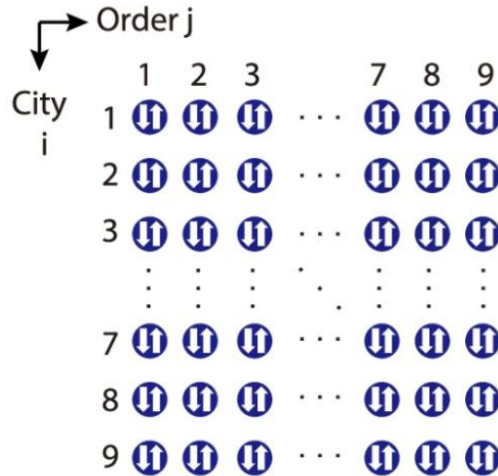


Figura 2.1 "Rappresentazione della Matrice di Spin $N \times N$ per il TSP"

Il sistema evolve spontaneamente per minimizzare la propria energia e per temperature sufficientemente basse, il sistema raggiunge il suo **stato fondamentale** (*Ground State*), ossia la configurazione di *spin* con l'energia più

bassa possibile. Dal punto di vista fisico, questo stato è la cosiddetta **fase ferromagnetica** (con l'allineamento di tutti gli *spin*).

Il modello di *Ising* fornisce un **linguaggio universale** per i problemi *NP-Hard*. Qualsiasi problema di ottimizzazione combinatoria può essere ricondotto alla ricerca del *Ground State* dell'Hamiltoniana di *Ising* mediante un processo di **mappatura**. Tale processo viene spesso espresso nella formulazione equivalente **Quadratic Unconstrained Binary Optimization (QUBO)**, che permette di sostituire le variabili di *spin* $\sigma_i \in \{-1, +1\}$ con variabili binarie $x_i \in \{0, 1\}$. La funzione obiettivo QUBO " $E(x)$ " è data da:

$$E(x) = \sum_{i < j} Q_{ij} x_i x_j + \sum_i C_i x_i + \text{costante}$$

La conversione di un problema *NP-Hard* in QUBO richiede l'incorporazione dei **vincoli** nella funzione obiettivo. Per raggiungere tale scopo, si aggiungono i **termini di penalità** quadratici:

$$E_{totale}(x) = E_{obiettivo}(x) + E_{vincoli}(x) = \sum \text{Costi} + \lambda \sum \text{Violazioni}^2$$

Il coefficiente di penalità (λ) deve essere calibrato con estrema precisione in quanto un λ troppo piccolo conseguirebbe soluzioni che violano i vincoli, mentre un λ troppo grande farebbe sì che la funzione di penalità domini l'Hamiltoniana. In quest'ultimo caso, il sistema sarebbe spinto a cadere nei minimi locali che non corrispondono alla soluzione ottimale.

La capacità di un'**Ising Machine** di risolvere un problema *NP-Hard* dipende quindi dalla precisione nel simulare la dinamica di un sistema di *Ising* che, grazie alla corretta mappatura in QUBO, possa raggiungere il *Ground State*, minimizzando sia i costi che le violazioni dei vincoli.

2.1.1 Minimo Energetico vs Soluzione Algoritmica: Perché il "quasi Ground State" non garantisce una soluzione quasi-ottimale

Il grande vantaggio del modello di *Ising* è che la soluzione ottima di un problema *NP-Hard* (come il TSP) equivale allo **stato fondamentale** (*Ground State*)

dell'Hamiltoniana di *Ising*, ossia la configurazione di *spin* ad energia minima. Tuttavia, il panorama energetico di un problema combinatorio complesso, come un TSP composto da 70 nodi, è tutt'altro che facilmente risolvibile.

La funzione energia QUBO definisce un paesaggio topografico estremamente articolato dove il numero di possibili soluzioni è esponenziale e la superficie energetica è costellata da innumerevoli **minimi locali**. Questi ultimi sono delle configurazioni di *spin* che rappresentano soluzioni ammissibili (ovvero soddisfano i vincoli del TSP) ma hanno un costo di percorso maggiore rispetto all'ottimo globale.

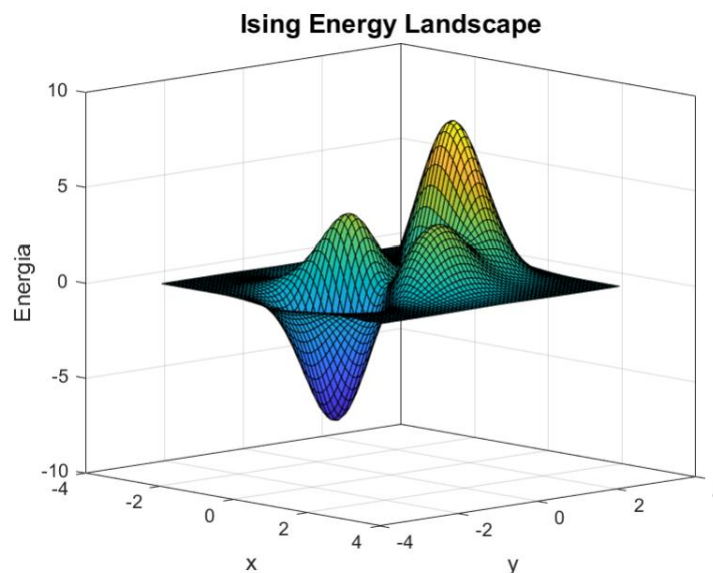


Figura 2.2 "Paesaggio Energetico di un'Hamiltoniana di Ising"

Quando un algoritmo di ottimizzazione tenta di minimizzare l'energia, esso segue un percorso in "discesa". In tale ambito, i metodi deterministici (come la discesa del gradiente o le macchine CIM) sono estremamente efficienti, ma una volta raggiunto un minimo locale, il sistema si **blocca**, mancando l'opportunità di trovare il vero *Ground State*, che potrebbe trovarsi oltre una barriera energetica più alta (come discusso nel *paper* scientifico "*Single Minimum Method*" di D. Xu e I. Kumazawa) [3]. In questo caso, il sistema ha trovato un **quasi Ground State** che, pur essendo stabile, non è la soluzione desiderata.

Per superare tali alte barriere energetiche, il sistema deve incorporare la **stocasticità** (o “rumore”). Questo concetto è alla base della tecnica di **annealing (ricottura)**, mutuata dalla metallurgia, dove un materiale viene riscaldato e poi raffreddato lentamente per raggiungere uno stato di più bassa energia. All'inizio del processo, il sistema è esposto ad un alto livello di rumore, conferendogli l'energia necessaria per saltare fuori dai minimi locali ed esplorare ampie regioni nello spazio delle soluzioni [4] . Successivamente, il rumore viene gradualmente ridotto, consentendo al sistema di assestarsi e convergere, con alta probabilità, verso il minimo energetico globale.

L'implementazione fisica del rumore è la principale differenza tra le architetture di *Ising Machine* (CIM vs pIM) e sarà il cuore dei prossimi capitoli, in particolare per la piattaforma spintronica **SMTJ**, che genera il rumore in modo intrinseco.

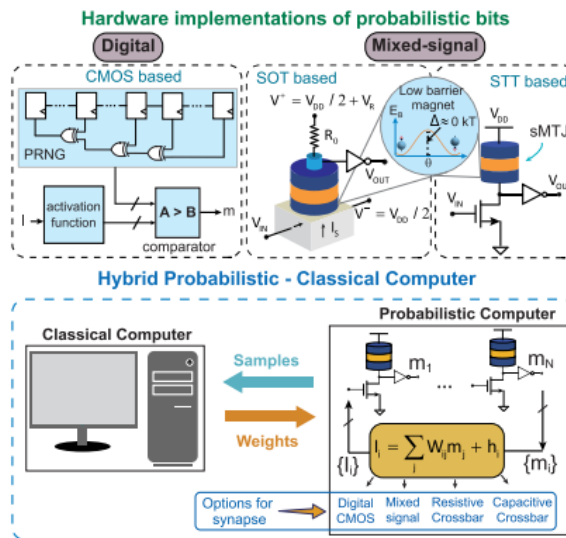


Figura 2.3 “Implementazioni Hardware del p-bit: digitale vs Mixed-Signal”

2.2 Paradigmi di Ising Machine a Confronto

Lo sviluppo del calcolo non convenzionale ha portato all'emergere di due principali paradigmi architetturali di *Ising Machine*, che si distinguono sia per il meccanismo fisico utilizzato per rappresentare lo *spin* che per la gestione della dinamica di minimizzazione energetica: le **Ising Machines Coerenti** e **Probabilistiche**.

2.2.1 Ising Machines Coerenti (CIM)

Le CIM sono paradigmi architetturali che eseguono la mappatura degli *spin* di *Ising* ($\sigma_i \in \{-1, +1\}$) alla **fase** o all'**ampiezza di un insieme di oscillatori accoppiati**. La minimizzazione dell'energia viene qui ottenuta attraverso l'evoluzione del sistema verso uno stato stabile a bassa energia. In particolare, le CIM sfruttano la dinamica di oscillatori accoppiati in un regime di **iniezione sub-armonica** (definita “**SHIL**”) [5] ; questo forzamento binaria la fase dell'oscillatore a valori discreti (0 o π radianti), rendendo la dinamica pari all'Hamiltoniana di *Ising*. Risulta importante notare che tale paradigma include le macchine basate su **Oscillatori Parametrici Ottici Degeneri (DOPD)** o le **Oscillator-based Ising Machines (OIM)**: queste ultime sono caratterizzate dall'utilizzo di reti di oscillatori elettronici/spintronici.

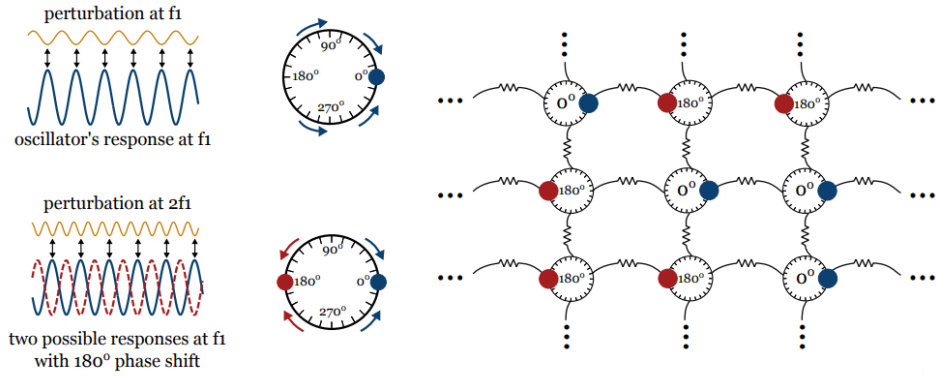


Figura 2.4: “Meccanismo di Binarizzazione di Fase nelle CIM (OIM)”

Una criticità riscontrabile è l'**eterogeneità di ampiezza**. Significa che la variazione delle ampiezze degli oscillatori accoppiati impedisce una mappatura accurata dell'Hamiltoniana, peggiorando così le prestazioni e intrappolando il sistema nei minimi locali. Per evitare ciò, sono state presentate soluzioni che usano reti neurali a *spike* con funzioni di trasferimento non lineari.

2.2.2 Ising Machines Probabilistiche (pIM)

Le pIM sono sistemi che implementano gli *spin* come variabili stocastiche, sfruttando l'**annealing termico** per l'esplorazione del complesso paesaggio energetico. Questi paradigmi attivano lo *spin* come un **p-bit**, ovvero un'entità che

alterna in modo intrinsecamente stocastico gli stati 0 e 1 (o +1 e -1). La probabilità di *flip* (viene così definito questo “alternarsi”) è determinata dalla funzione obiettivo $E(x)$ e da un parametro di **temperatura inversa** (β), giungendo a simulare la “statistica di *Boltzmann*”. Questo meccanismo di continuo aggiornamento, noto come **Gibbs Sampling**, va a simulare l'algoritmo di **Simulated Annealing (SA)** direttamente in *hardware* [4]. Si noti che il rumore facilita la fuga dai minimi locali, un limite intrinseco degli approcci puramente coerenti.

L'implementazione hardware più modernamente avanzata è basata sulle **Superparamagnetic Tunnel Junctions (SMTJ)**, che sfruttano il **rumore termico ambientale** ($k_b T$) come fonte **naturale** di fluttuazioni, riducendo i consumi e i costi per la generazione di tali casualità [6].

2.2.3 Analisi Critica: Prestazioni a Confronto

Dall'analisi critica delle prestazioni, anche a livello *software*, emergono vantaggi distinti tra i due paradigmi:

1. **Efficacia nella ricerca:** le pIM tendono a superare le CIM nel numero di iterazioni necessarie da compiere per raggiungere soluzioni quasi-ottimali. Tale comportamento è attribuibile alla capacità dei sistemi probabilistici di navigare il paesaggio energetico in modo più efficiente, sfruttando il rumore per evitare precoci situazioni di blocco;
2. **Efficienza energetica:** il paradigma architetturale pIM (soprattutto con implementazione SMTJ) offre una prospettiva di **efficienza energetica superiore**, dell'ordine dei *femtojoule* per operazione;
3. **Velocità potenziale:** le CIM o OIM basate su oscillatori ad alta frequenza (dell'ordine dei GHz) hanno un potenziale di velocità di calcolo ultraveloce, a patto che non si riscontrino problemi di coerenza e di eterogeneità.

In conclusione, le CIM mostrano un potenziale di velocità legato in particolare alla frequenza di oscillazione, mentre le pIM si dimostrano più adatte nell'evitare i minimi locali, con un netto vantaggio di efficienza energetica specialmente grazie all'*hardware* spintronico [7].

2.3 La Piattaforma Spintronica SMTJ: Il p -bit come spin stocastico implementato con SMTJ e l'origine del rumore intrinseco

L'efficienza energetica delle *Probabilistic Ising Machines* dipende innanzitutto dalla realizzazione fisica del p -bit, ossia il componente che funge da generatore sintonizzabile di numeri casuali binari. La piattaforma più promettente e avanzata in questo ambito è sicuramente quella basata sulla **spintronica**, in particolare attraverso l'uso delle *Superparamagnetic Tunnel Junctions (SMTJ)*.

Una SMTJ è un dispositivo a giunzione magnetica a tunnel con una scelta di *design* ottimizzato in cui lo "strato libero" (*free layer*) presenta una barriera energetica relativamente bassa. Da sottolineare come proprio il fenomeno del **superparamagnetismo** consenta alla magnetizzazione dello strato libero di oscillare stocasticamente tra due stati di resistenza distinti: "**Resistenza Parallela**" (P) e "**Resistenza Anti-Parallela**" (AP). Quest'ultimi sono conseguenza della sola energia termica ambientale ($k_b T$) e a ciascuno viene associato lo *spin* binario del p -bit (± 1 o $0/1$). Infatti, la dinamica di inversione stocastica della SMTJ implementa di per sé l'equazione del p -bit [8] .

È importante notare che la caratteristica distintiva delle SMTJ è l'utilizzo del **rumore termico ambientale** come fonte intrinseca di casualità per gestire le fluttuazioni di *spin*. A differenza delle implementazioni CMOS, tale approccio innovativo elimina la necessità di complessi circuiti digitali (come i generatori di numeri pseudo-casuali, PRNG). Inoltre, esso permette di ridurre drasticamente il consumo energetico legato alla generazione di casualità, con un consumo energetico stimato a livello di *femtojoule* per operazione.

Infine, le SMTJ sono in grado di effettuare i *flip* nell'ordine del **nanosecondo**, evidenza significativa della loro superiorità energetica rispetto alle architetture convenzionali, superando anche il limite di consumo del *Von Neumann Bottleneck*.

Un passo cruciale per la ricerca futura sarà la realizzazione del p -bit con SMTJ, che assicurerà elevatissima efficienza e scalabilità [8] . Questa tecnologia è inoltre

pienamente **compatibile con i processi CMOS** e con le architetture di memoria già esistenti [9].

2.4 Architetture per la Velocità: Concetto di Gibbs Sampling (sincrono, pseudo-asincrono) e Sparsificazione per il parallelismo

La notevole potenza computazionale delle **pIM** non risiede soltanto nell'efficienza del singolo ***p-bit***, ma anche nella capacità di aggregare milioni di *p-bits* per eseguire il calcolo in modo **contemporaneamente parallelo**. Alla base della dinamica di minimizzazione energetica risiede il ***Gibbs Sampling***, uno schema iterativo che guida il sistema verso il suo *Ground State*, ossia lo stato di equilibrio termodinamico.

La sua velocità di esecuzione è però limitata dalla necessità che gli *spin* interconnessi si aggiornino sequenzialmente. In merito a quest'ultimo problema, l'aggiornamento di uno *spin* dipende dallo stato di tutti i suoi vicini, creando un legame di dipendenza sequenziale che rallenta il sistema ad una velocità $O(N)$ (con N pari al numero di *spin*).

Per evitare ciò, le architetture moderne di *p-computing* sfruttano le proprietà del grafo di connessione definendo due principali tipologie di *Gibbs Sampling*:

- ***Gibbs Sampling* sincrono**: la soluzione consiste nell'aggiornare tutti gli *spin* uno alla volta, richiedendo N cicli per uno *sweep* completo;
- ***Gibbs Sampling* pseudo-asincrono**: tale schema iterativo sfrutta la caratteristica che gli *spin* non direttamente connessi possono essere aggiornati simultaneamente. Tale parallelizzazione è ottenuta tramite la tecnica di **colorazione del grafo** (*Graph Coloring*). Consiste nell'assegnare lo stesso "colore" agli *spin* non interconnessi e successivamente aggiornare in parallelo i singoli colori con *clock* sfasati, riducendo la complessità da $O(N)$ a $O(C)$, dove C è il numero di colori (molto minore di N).

2.4.1 Architetture per la Scalabilità (Cross-bar Array e Sparsificazione)

L'impressionante risultato della scalabilità delle pIM a milioni di *spin* è stato raggiunto grazie ai progressi nell'ingegneria *hardware* e all'ottimizzazione del grafo di connessione.

I sistemi pIM sono implementati utilizzando architetture a *Cross-bar Array*: tale struttura, ereditata dalle memorie MRAM, integra le SMTJ con i circuiti CMOS periferici. Il vantaggio principale è che il *cross-bar array* implementa una nuova tecnologia mediante l'elaborazione "*in-memory*", realizzando le somme di accoppiamento ($\sum J_{ij} \sigma_i$) direttamente sul *chip* [8]. Questo permette di eliminare la maggior parte del *data shuttling* tra CPU e memoria, superando il *Von Neumann Bottleneck* e consentendo densità molto elevate.

Tuttavia, per risolvere problemi che presentano un denso grafo di connettività (proprio come nel caso della matrice $N^2 \times N^2$ del TSP), è necessario ridurre la densità per abilitare il parallelismo e un *synapse time* più breve. Ciò perché calcolare $\sum J_{ij} \sigma_i$ (la somma delle interazioni) richiede tempo, rallentando il *Gibbs Sampling*. La cosiddetta tecnica di sparsificazione [10] si realizza effettuando delle repliche degli *spin* critici (*ferromagnetic COPY gates*) e distribuendo le connessioni originali tra queste copie. Nonostante il numero totale di *p-bits* aumenta, si riduce drasticamente la densità locale del grafo e il tempo di calcolo sinaptico [10].

Capitolo 3:

Il TSP e la Critica dei Metodi Algoritmici Tradizionali

3.1 Rassegna delle Soluzioni Classiche al TSP: Algoritmi Esatti (Branch and Bound) e Metaeuristici (Genetic Algorithm)

Il *Traveling Salesman Problem*, data la sua definizione di problema *NP-Hard* per eccellenza, ha favorito lo sviluppo di due principali macrocategorie di metodologie risolutive: gli algoritmi **esatti** e le **metaeuristiche**. La distinzione alla base tra i due metodi risulta essere la “certezza” matematica del raggiungimento dell'ottimo globale. Resta tuttavia invariata la necessità di ottenere soluzioni di alta qualità nel minor tempo computazionale possibile.

3.1.1 Metodi Algoritmici Esatti (Branch and Bound)

La prima categoria di metodologie algoritmiche risolutive racchiude i **metodi esatti** che, come da definizione, garantiscono di trovare la **soluzione ottimale globale** per ogni istanza del problema (indipendentemente dalla sua complessità). Tuttavia, quest'ultima risulta limitare la loro generale applicabilità, a causa della natura esponenziale del tempo di esecuzione.

Il **Branch and Bound (B&B)** risulta essere il metodo esatto più efficiente per il TSP e altri problemi di ottimizzazione combinatoria (come problemi di *flow-shop scheduling*) [4]. Esso opera esplorando lo spazio delle soluzioni, visualizzato come un vero e proprio albero di ricerca, e consta di due fasi principali:

- **Branch (Ramificazione):** questo primo stadio permette di suddividere il problema di partenza in “sotto-problemi” più piccoli, che tendono ad espandere l'albero di ricerca;
- **Bound (Delimitazione):** il secondo stadio del metodo prevede l'utilizzo di funzioni di costo per stimare il valore minimo che si può ottenere da un singolo sotto-problema. Se il limite inferiore appena calcolato è peggiore

della migliore soluzione trovata fino a quel momento, l'intero ramo dell'albero viene **tagliato (potato)**, evitando un'ulteriore vana esplorazione e riducendo drasticamente il tempo di ricerca rispetto all'uso della forza bruta (*brute force*).

Da notare però che, nonostante l'efficienza di questo “meccanismo di taglio”, la complessità computazionale del B&B rimane **esponenziale** rispetto alla dimensione del problema ($O(N!)$ o superiore). Ciò significa che per istanze di TSP con N alquanto grande, il tempo necessario per il ***Time to Solution* (TTS)** diventa proibitivo e di conseguenza insostenibile.

3.1.2 Metaeuristiche e Algoritmi Ispirati alla Natura (Genetic Algorithm)

Altre metodologie risolutive sono le **metaeuristiche**, ossia strategie di ricerca a livello superiore che indirizzano gli algoritmi di ottimizzazione. Il loro obiettivo principale è il bilanciamento dell'esplorazione in zone non ancora visitate e del perfezionamento delle soluzioni note.

Il ***Genetic Algorithm* (GA)** è una delle metaeuristiche più note e si ispira direttamente ai processi di selezione naturale e genetica. Si concentra principalmente su una popolazione di soluzioni candidate (cromosomi), applicando operatori di **selezione**, **crossover** e **mutazione**. Questi eseguono rispettivamente la scelta degli individui con la *fitness* migliore, la combinazione delle porzioni di due soluzioni “genitore” per crearne di nuove e potenzialmente superiori ed infine l'introduzione di alterazioni stocastiche per esplorare punti inesplorati dello spazio delle soluzioni.

Il GA conferma di essere una metaeuristica eccellente dato l'intrinseco parallelismo e la sua caratteristica di esplorare spazi di soluzioni vasti e complessi. La possibilità di utilizzarlo per manipolare direttamente le configurazioni, ne ha garantito un ampio utilizzo nel TSP.

Tuttavia, lo svantaggio maggiore da sottolineare è che i GA **non garantiscono** il raggiungimento della soluzione ottimale a livello globale. Tutto ciò a causa della loro natura probabilistica, che unita all'uso delle strategie *greedy* (avide) e alla

alta dipendenza dai parametri iniziali, li rende suscettibili alla **convergenza prematura** [8] [11].

3.2 Analisi Critica delle Strategie Greedy: Vantaggi di velocità e Svantaggi di qualità

Per ottenere un notevole vantaggio in termini di **Time to Solution (TTS)** e di **semplicità implementativa** nella risoluzione di problemi *NP-Hard*, molte metaeuristiche adottano **strategie Greedy** ("avide") [11]. Nonostante siano convenienti a livello computazionale, tali strategie presentano alla base un difetto "strutturale": il sacrificio della **qualità** e della stabilità della soluzione ottenuta.

Infatti, il metodo *greedy* consiste nella continua scelta, ad ogni passo, dell'opzione che appare localmente migliore, senza però considerare le conseguenze a lungo termine. Ciò rende tale approccio molto rapido (specialmente durante le fasi di distruzione e ricostruzione in algoritmi come l'*Iterated Greedy* (IG)) ma la sua natura mira solo ad un'ottimizzazione locale.

3.2.1 Rischio di Convergenza Prematura e l'Incapacità delle Euristiche di Fuggire dai Minimi Locali

La principale criticità delle strategie *greedy* adottate da metaeuristiche è l'elevato rischio di **convergenza prematura** e la difficoltà di evitare il potenziale blocco nei **minimi energetici locali**.

La convergenza precoce si verifica quando l'algoritmo si stabilizza velocemente su una soluzione sub-ottimale nel paesaggio energetico ed interrompe la ricerca prima di esplorare regioni potenzialmente migliori, in cui potrebbe trovarsi il vero **Ground State**. I metodi *greedy* tendono a seguire dei percorsi in "discesa" nel paesaggio energetico e quando raggiungono un minimo energetico locale, l'algoritmo non dispone di un meccanismo in grado di incrementare temporaneamente l'energia ($\Delta E > 0$), dato che il gradiente è nullo. Tale barriera energetica impedisce la possibilità del sistema di ricercare soluzioni globalmente migliori.

Anche gli algoritmi più sofisticati, come il **Genetic Algorithm (GA)**, che combinano strategie di affinamento delle soluzioni locali con l'esplorazione globale del paesaggio energetico (i cosiddetti algoritmi ibridi), presentano tali difetti. Seppur il *crossover* e la mutazione permettano una certa diversificazione delle soluzioni, le procedure di ottimizzazione locale tendono a spingere il sistema troppo rapidamente verso il minimo locale, rendendone difficile la fuga.

In conclusione, l'approccio con le strategie *greedy* **non rappresentano un meccanismo stocastico o termico sufficientemente efficace** per risolvere la topologia complessa di problemi di ottimizzazione. Anche se la soluzione verrà trovata in tempo polinomiale, questa sarà comunque di qualità insufficiente. È questo insuccesso combinato che porta alla ricerca di paradigmi di calcolo che possano integrare l'*annealing* come proprietà fisica intrinseca, consentendo di evitare i limiti imposti dalla convergenza deterministica tradizionale.

3.3 Simulated Annealing (SA) Algoritmico: Il Precursore Stocastico

Il **Simulated Annealing (SA)**, introdotto nell'ambito informatico da *Kirkpatrick, Gelatt e Vecchi* nel 1983 [2] , è il precursore stocastico che ha permesso l'integrazione del **rumore controllato** nei processi di ottimizzazione combinatoria. L'SA si basa sul "**Criterio di Metropolis**" che accetta temporaneamente passaggi verso stati ad energia superiore ($\Delta E > 0$), permettendo al sistema di fuggire dai minimi locali [2] . Il controllo del rumore è interamente affidato al parametro della **temperatura** (T).

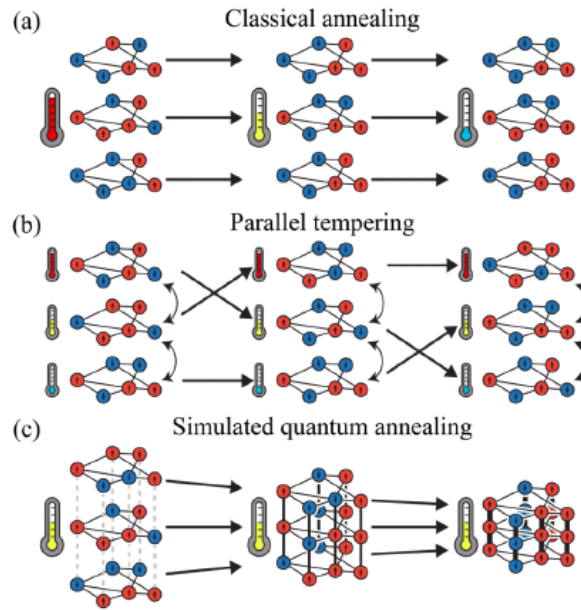


Figura 3.1: “Rappresentazione schematica dei meccanismi dinamici di annealing utilizzati per la ricerca parallela del Ground State”

3.3.1 Dipendenza dalla Curva di Raffreddamento e Parametri Critici

L'efficienza del *Simulated Annealing* è dipendente dalla precisa calibrazione dello **schema di raffreddamento** (*Cooling Schedule*), che determina l'evoluzione della temperatura T nel tempo [2]. I parametri critici di tale schema sono:

1. **La temperatura iniziale (T_0):** viene scelta in modo tale che il sistema si trovi inizialmente in una fase di **esplorazione** quasi stocastica. Questo perché una T_0 troppo bassa riduce drasticamente la probabilità ($e^{-\frac{\Delta E}{T_0}}$) di accettare passi “peggiorativi”, e si avrebbe la degenerazione del *Simulated Annealing* in un'euristica *greedy* che si bloccherebbe al primo minimo energetico locale;
2. **Il fattore di raffreddamento (α):** esso determina la velocità con cui T viene ridotta ad ogni passo (definendo l'*inner loop*), tipicamente con una legge esponenziale del tipo: $T_{k+1} = \alpha \cdot T_k$ (con α compreso tra 0.8 e 0.99);
3. **La lunghezza dell'iterazione (*Markov Chain Length*):** è il numero di aggiornamenti di *spin* da eseguire per ogni valore di temperatura T . Questo valore deve essere abbastanza grande da permettere così al sistema di

raggiungere uno stato di **equilibrio termico** prima di raffreddarsi ulteriormente.

Tuttavia, la critica fondamentale mossa all'SA riguarda il **compromesso prestazionale tra la “garanzia teorica” e la “fattibilità pratica”**.

Si dimostra che per **garantire la convergenza** alla soluzione ottimale globale con probabilità massima, la temperatura deve scendere in modo **asintoticamente lento** (ad esempio, con una legge logaritmica: $T_k \propto \frac{1}{\log(k)}$). Ma nella pratica, un *Cooling Schedule* così lento condurrebbe ad un **Time to Solution inaccettabile**, tendente ad infinito. L'utilizzo di schemi di raffreddamento più rapidi, impone però il rischio di **quenching** (*congelamento*) per cui il sistema si stabilizzerebbe prematuramente in un **minimo energetico locale** di bassa qualità.

La dipendenza dalla calibrazione di T_0 , α e della lunghezza della catena di iterazione è un pesante vincolo a livello algoritmico che limita l'adattabilità dell'SA a diversi problemi di ottimizzazione. Data la sua natura da algoritmo **intrinsecamente sequenziale** (il criterio di *Metropolis* impone che un solo *spin* si aggiorni alla volta), ne limita drasticamente la velocità di calcolo anche su potenti *hardware* convenzionali. Questo problema ha sviluppato la ricerca di **hardware non convenzionali**, come le **Probabilistic Ising Machines (pIMs)**, che implementano il campionamento stocastico in modo **contemporaneamente parallelo** (come il rumore termico delle SMTJs).

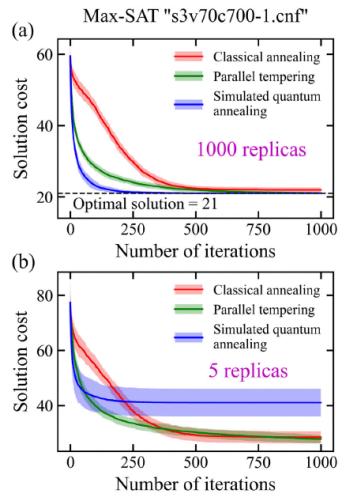


Figura 3.2: “Confronto prestazionale tra Classical Annealing, Parallel Tempering e Simulated Quantum Annealing in funzione del numero di repliche”

Capitolo 4:

Implementazione Critica: TSP, Parametri e Trade-off Prestazionale

Questo capitolo si focalizza sull'implementazione pratica e la valutazione critica delle **Ising Machines Probabilistiche (pIM)** a base **SMTJ** (*Superparamagnetic Tunnel Junction*) per risolvere il **Traveling Salesman Problem (TSP)** [16].

Dopo aver stabilito i fondamenti teorici (Capitolo 2) e criticato le inefficienze dei metodi algoritmici classici (Capitolo 3), ora l'obiettivo risulta essere duplice:

1. Formalizzare il processo di traduzione del TSP nel modello **QUBO** (**Quadratic Unconstrained Binary Optimization**), inclusa la critica alla calibrazione del coefficiente di penalità λ , che governa la validità della soluzione;
2. Valutare l'efficacia dell'architettura SMTJ-pIM nell'ottimizzare il rapporto tra **Time to Solution (TTS)** e **Accuratezza della soluzione**. Tale analisi dimostra, attraverso metriche di efficienza energetica, come il **Global Annealing** accoppiato al rumore intrinseco dell'*hardware* superi i limiti di velocità e consumo delle soluzioni di *annealing* classiche.

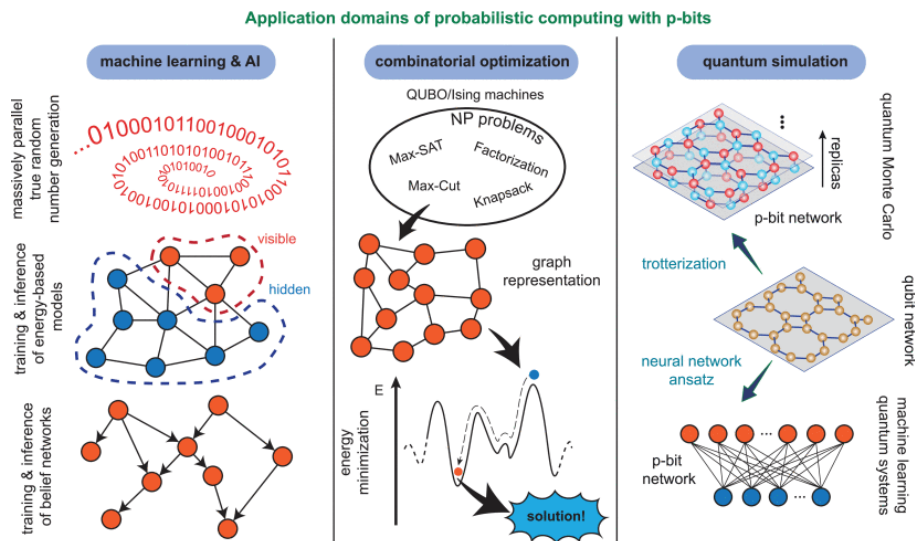


Figura 4.1: "La visione Full-Stack del calcolo probabilistico e la sua applicazione a problemi NP-Hard"

4.1 Mappatura del TSP sull'Hamiltoniana: Formulazione dei Termini di Vincolo e Costo

La fase ingegneristicamente più importante per la risoluzione del *Traveling Salesman Problem* mediante un'*Ising Machine* (IM) consiste nella sua rigorosa trasposizione nel formalismo **Quadratic Unconstrained Binary Optimization** (QUBO) [18]. Tale convenzionalismo è l'equivalente matematico dell'Hamiltoniana di *Ising*. Esso richiede di convertire i vincoli e la funzione obiettivo di costo del TSP in un'unica funzione quadratica di energia che dovrà essere ovviamente minimizzata, poiché rappresenta fisicamente lo stato di equilibrio del sistema.

4.1.1 Definizione delle Variabili di Spin

Il *Traveling Salesman Problem* (TSP) può essere mappato su di un grafo completamente connesso, nel quale l'obiettivo è individuare il percorso chiuso più breve che consenta di visitare tutte le N città esattamente una ed una sola volta, tornando infine al punto di partenza. Ne consegue che per modellare un problema con N città, sia necessario introdurre una matrice di $N \times N$ **spin binari** [15]. Si definisce con $S_{i,j}$ la variabile di spin, dove i rappresenta la **città** ($i \in \{1, 2, \dots, N\}$) e j invece la **posizione** nel percorso ($j \in \{1, 2, \dots, N\}$); queste variabili possono assumere solo due valori: $S_{i,j} \in \{0, 1\}$. La configurazione $\mathbf{S} = \{S_{i,j}\}$ che minimizza l'energia del sistema (rappresentata dalla funzione Hamiltoniana) è la soluzione ottimale del problema.

L'Hamiltoniana totale $H_{TSP}(\mathbf{S})$ si compone di due contributi principali e distinti: il **termine di costo** (H_{Costo}), ossia la funzione obiettivo, e il **termine di vincolo** ($H_{Vincoli}$), che garantisce la validità del percorso. Tali due termini vengono sommati e pesati dal coefficiente di penalità " λ " [14]:

$$H_{TSP}(\mathbf{S}) = H_{Costo}(\mathbf{S}) + \lambda \cdot H_{Vincoli}(\mathbf{S})$$

Lo stato ad energia minima, detto *Ground State*, corrisponderà quindi al cammino ottimale che rispetta i vincoli imposti [17].

4.1.2 Formulazione dei Termini di Vincolo ($H_{Vincoli}$)

Il termine $H_{Vincoli}$ è il componente di penalità che garantisce che lo stato finale della configurazione di spin sia una soluzione **valida** del TSP: ogni città dovrà apparire una ed una sola volta. Questo termine, per i problemi *NP-Hard* mappati su QUBO, viene costruito sfruttando due vincoli di uguaglianza, come formalizzato nel *paper* scientifico “*Ising formulations of many NP problems*” di A. Lucas et al. [18].

Il primo vincolo di uguaglianza è $H_{Vincoli,Città}$ che impone che ogni città i sia visitata **esattamente una volta** in una qualsiasi posizione j del percorso. La somma matematica degli *spin* su tutte le posizioni per una data città i , deve essere pari ad 1:

$$H_{Vincoli,Città} = \sum_{i=1}^N \left(\sum_{j=1}^N S_{i,j} - 1 \right)^2$$

Se per una data città i , la somma risulta essere 0 o maggiore di 1, il termine quadratico risulterà positivo, contribuendo all'energia di penalità. Questi due casi avvengono rispettivamente quando la città non viene visitata o quando viene visitata più volte. Solo quando $\sum_{j=1}^N S_{i,j} = 1$, il termine $H_{Vincoli,Città}$ sarà nullo.

Invece, il secondo vincolo di uguaglianza $H_{Vincoli,Ordine}$, impone che ogni posizione j del percorso sia occupata da **esattamente una città** i . Analogamente al caso di prima, anche qui la somma degli spin su tutte le città i per una data posizione j deve essere uguale ad 1.

$$H_{Vincoli,Ordine} = \sum_{j=1}^N \left(\sum_{i=1}^N S_{i,j} - 1 \right)^2$$

Questo termine risulterà nullo solo quando la posizione j sarà occupata da una ed una sola città.

Il termine di vincolo totale risulta la somma di questi due contributi:

$$H_{Vincoli} = H_{Vincoli,Città} + H_{Vincoli,Ordine}$$

La possibile espansione di questi termini quadratici potrebbe generare un insieme di **accoppiamenti** tra i p -bit [15] . Ad esempio, $(\sum_j S_{i,j})^2$ che si espande in una serie di termini $S_{i,j}, S_{i,k}$ (per $j \neq k$); questi valori rappresentano le interazioni che renderebbero i p -bit incompatibili, ossia che solo uno dei due possa essere attivo in una riga o colonna.

4.1.3 Formulazione del Termine di Costo (H_{Costo})

Per definire la funzione obiettivo da minimizzare, che corrisponde alla distanza complessiva del cammino ottimizzato, ci si riferisce al termine H_{Costo} . Assumendo che $d_{i,l}$ sia la distanza tra la città i e la città l , il vincolo di costo è pari alla somma di tutte le distanze tra le città consecutivamente adiacenti:

$$H_{Costo}(S) = \sum_{i=1}^N \sum_{l=1}^N \sum_{j=1}^N d_{i,l} \cdot S_{i,j} \cdot S_{l,j+1}$$

dove:

- $d_{i,l}$ rappresenta il “peso” dell'accoppiamento tra le città i e l ;
- il prodotto $S_{i,j} \cdot S_{l,j+1}$ è invece il termine di accoppiamento quadratico che è pari a 1 (per cui non nullo) solo quando la città i occupa la posizione j e la città l occupa la posizione immediatamente successiva $j+1$.

Risulta di fondamentale rilevanza notare che la periodicità del TSP (ossia che si torni al punto di partenza iniziale) è implicita nella sommatoria per la posizione finale $j = N$, dove per $j + 1$ ci si riferisce alla posizione 1.

H_{Costo} tende a generare delle interazioni che favoriscono l'accoppiamento di città con una distanza $d_{i,l}$ minima in posizioni consecutive nel percorso, facendo sì che il sistema sia guidato verso la minima lunghezza possibile.

4.1.4 L'Hamiltoniana Finale e la Calibrazione (TSP vs CTSP)

Una fase fondamentale nella **progettazione della funzione Hamiltoniana** riguarda la scelta del coefficiente di penalità λ (indicato anche come A o w nei *paper* scientifici di riferimento, come in “*Energy-efficient superparamagnetic Ising machine and its application to traveling salesman problems*” di Yang et al.). In

effetti un'errata "calibrazione" di λ porta a dei fallimenti sistematici nel processo di convergenza al *Ground State*, indipendentemente dalla potenza di calcolo della *Ising Machine* in questione.

Il termine di penalità " $\lambda \cdot H_{vincoli}$ " dominerà il termine di costo in caso di violazione di un vincolo, per garantire che la soluzione non valida abbia un'energia **superiore** alla soluzione ottimale. Tale condizione di dominanza necessita che il coefficiente di penalità sia maggiore della massima differenza di costo che il sistema potrebbe generare. Una calibrazione molto utilizzata è rappresentata dalla formula:

$$\lambda > N \cdot \max(d_{i,l})$$

dove " $N \cdot \max(d_{i,l})$ " rappresenta una stima superiore del **costo totale** di qualsiasi altro percorso.

Tuttavia, la problematica principale da risolvere non riguarda solo garantire la dominanza, ma anche ottimizzare λ per l'efficienza di calcolo, in quanto possono verificarsi due casistiche:

1. **λ troppo basso ($\lambda \ll \lambda_{ottimale}$):** qui il sistema non riesce a penalizzare abbastanza le violazioni dei vincoli e l'energia dei percorsi non validi è vicina a quella dei percorsi buoni. Il sistema tende ad accontentarsi di soluzioni non valide ma che comunque presentino un H_{Costo} basso. Crolla la **success rate** di trovare una soluzione valida;
2. **λ troppo alto ($\lambda \gg \lambda_{ottimale}$):** d'altro canto, qui i vincoli dominano in modo troppo eccessivo, appiattendolo le differenze di energia tra i vari percorsi validi. Di conseguenza, il sistema ha molta difficoltà a distinguere un percorso più corto da altri percorsi validi, ma leggermente più lunghi. Tutto ciò rende il paesaggio energetico piatto per le soluzioni valide, rallentando la fase di convergenza verso il *Ground State*, aumentando il **Time to Solution (TTS)**.

Pertanto, la calibrazione perfetta di λ è una vera e propria ricerca di equilibrio che massimizzi la differenza di energia tra il *Ground State* e il primo stato eccitato

valido, e che mantenga al contempo un grande divario energetico tra le soluzioni valide e non. Questo processo di calibrazione è un esempio di come l'ottimizzazione algoritmica sia cruciale per massimizzare le prestazioni di un *hardware* di calcolo non convenzionale (pIM).

4.2 Strategie Dinamiche e Ruolo del Rumore: L'Annealing come guida nell'esplorazione del paesaggio energetico

Risulta importante ribadire come la risoluzione dei problemi *NP-Hard*, richieda che l'algoritmo non si blocchi “prematuramente” nei **minimi locali** del paesaggio energetico esplorato. Se algoritmi deterministici come la discesa del gradiente o le già citate “euristiche *Greedy*” patiscono particolarmente questa problematica, le *Probabilistic Ising Machines* (pIM) sfruttano la **stocasticità (rumore)** intrinseca. Questo approccio, noto come *annealing*, permette di evitare il problema guidando il sistema attraverso un'esplorazione termica verso il *Ground State* (ottimo globale).

4.2.1 Il Ruolo Centrale del Rumore Stocastico

Nelle architetture SMTJ-pIM, il **rumore** risulta essere la **risorsa computazionale** essenziale: la piattaforma, infatti, sfrutta la fisica dei vari materiali per generare stocasticità ad un basso costo energetico.

Più nello specifico, le ***Superparamagnetic Tunnel Junctions* (SMTJ)** sono dispositivi spintronici in cui lo strato libero (il “*free layer*”) è predisposto ad operare in un regime dove la barriera energetica è particolarmente bassa. L'**energia termica ambientale** ($k_b T$), a temperatura ambiente, è sufficiente a causare delle casuali fluttuazioni nella magnetizzazione che porta ad un'oscillazione continua dello spin tra gli stati di resistenza “Parallela” (P) e “Anti-Parallela” (AP). A ciascuno di questi stati è associato a uno spin binario ± 1 (o 0/1). Tale processo di inversione stocastica nella SMTJ consente di implementare l'equazione del *p-bit*, che, a sua volta, ricalca la logica di accettazione del **criterio di Metropolis** già citato riguardo il *Simulated Annealing*. Questo meccanismo consente al sistema di effettuare delle transizioni verso stati ad energia superiore

($\Delta E > 0$), conferendo tutta l'energia utile per “**saltare fuori**” dalle **barriere dei minimi locali**. Sfruttare questo rumore termico intrinseco (*intrinsic annealing*) permette di continuare la ricerca del *Ground State* ed elimina la necessità di complessi circuiti digitali per introdurre la stocasticità. Tale accorgimento consegue una riduzione del consumo energetico legato alla generazione di casualità e contribuisce ad un'efficienza energetica dell'ordine dei **femtojoule (fJ)** per operazione.

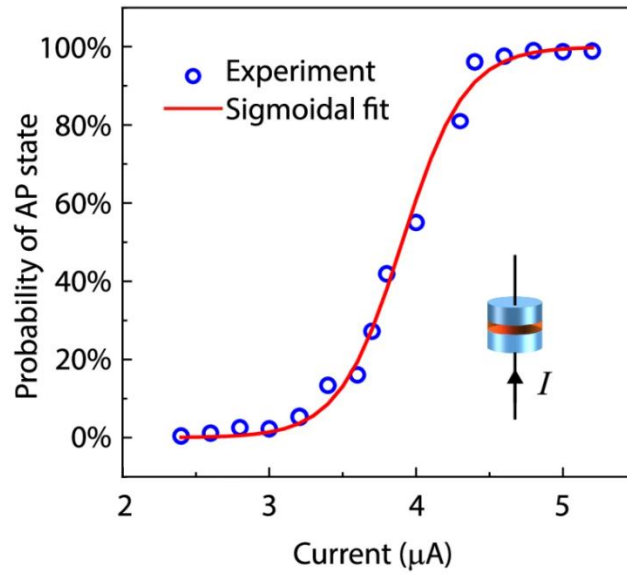


Figura 4.2: “Probabilità di transizione stocastica del p-bit SMTJ”

4.2.2 Schemi Dinamici di Annealing a Confronto

Tuttavia, affinché l'*hardware* SMTJ-pIM possa ricavare la soluzione ottimale con maggiore probabilità, alla **stocasticità intrinseca** (*intrinsic annealing*) deve essere combinato un meccanismo di **Annealing Schedule** globale (*global annealing*) [14]. Tale processo controlla e modula il rumore o la probabilità di *flip* nel tempo, basandosi su schemi algoritmici derivanti dalla fisica statistica.

Si dimostra che l'efficienza del sistema è funzione della sua capacità di implementare versioni perfezionate di questi schemi:

- **Classical Annealing (CA):** questo primo schema necessita che l'inverso della temperatura efficace ($c = \frac{1}{k_b T}$) aumenti gradualmente nel tempo, costringendo il sistema a stabilizzarsi nel *Ground State*. Sebbene la

convergenza allo stato fondamentale possa essere garantita solo se la velocità di raffreddamento è **asintoticamente lenta**, tale approccio è nella realtà impraticabile a causa di un ***Time to Solution (TTS) non accettabile***. Qui l'ostacolo da superare è calibrare un raffreddamento non-lineare abbastanza rapido ma che eviti il fenomeno del *quenching* (blocco precoce in un minimo locale);

- ***Simulated Annealing (SA) algoritmico***: su architetture convenzionali, l'SA viene implementato basandosi su uno schema di ***Gibbs Sampling***. Questo approccio, tuttavia, è **intrinsecamente sequenziale**, limitando di gran lunga la velocità di calcolo ad un valore $O(N)$, dove N è il numero di *spin*. Si aggiunge a questa limitazione anche la richiesta di risorse significative per generare algoritmicamente le casualità (PRNG);
- ***Parallel Tempering (PT)***: questo schema algoritmico utilizza invece più repliche del sistema che evolvono **in parallelo** a diverse temperature ($T_1 < T_2 < \dots < T_M$). L'inversione periodica di configurazioni tra repliche (ossia uno *swap* tra repliche a temperatura alta e bassa) consente alle repliche più "calde" di superare le barriere energetiche, migliorando l'esplorazione e aumentando la ***success rate*** a bassa temperatura;
- **SMTJ-pIM (implementazione ibrida)**: l'approccio spintronico, citato prima, si presenta come un modello **ibrido** che combina l'***Annealing Intrinseco*** (sempre attivo grazie al rumore termico intrinseco dell'SMTJ) con l'***Annealing Globale***. La dinamica del sistema emula il *Gibbs Sampling* attraverso una logica di campionamento **sincrona o pseudo-asincrona**.
 1. ***Gibbs Sampling sincrono***: consiste nell'aggiornamento degli *spin* uno alla volta, richiedendo N cicli per uno *sweep* completo;
 2. ***Gibbs Sampling Pseudo-Asincrono***: sfruttando la proprietà degli *spin* non direttamente interconnessi, che possono essere aggiornati simultaneamente, si utilizza la tecnica della **colorazione del grafo** (*Graph Coloring*). Ciò diminuisce la complessità da $O(N)$ ad $O(C)$, dove C è il numero di "colori" (molto minore di N), conferendo un alto grado di parallelismo al sistema.

Questa combinazione *hardware-software* consente alla SMTJ-pIM di operare a velocità molto elevate, presentando un netto vantaggio di **efficienza energetica** per l'implementazione stocastica.

Nel contesto del *Travelling Salesman Problem*, la calibrazione esatta dell'*Annealing* globale è un punto cruciale per superare la topologia complessa del paesaggio energetico. Infatti, la temperatura iniziale " T_0 " deve essere **sufficientemente alta** da garantire che l'energia termica superi qualsiasi barriera energetica iniziale ($\Delta H = H_{max} - H_{min}$) all'interno del sistema, consentendo al sistema di esplorare lo spazio delle soluzioni. Se T_0 fosse troppo bassa, l'algoritmo finirebbe per degenerare in un'euristica *Greedy*. Al contrario, la temperatura finale " T_{Finale} " deve essere **sufficientemente bassa** da bloccare il *Ground State*, eliminando le varie fluttuazioni che potrebbero far saltare il sistema fuori dalla soluzione ottimale.

Nel paper scientifico "*Energy-efficient superparamagnetic Ising machine and its application to traveling salesman problems*" di Yang et al., si dimostra che una **strategia lineare di annealing** (ossia dove la T aumenta linearmente) può garantire la convergenza ma solo ottimizzando l'*annealing schedule*, necessaria per ridurre il numero di iterazioni di calcolo.

La concomitanza di *Intrinsic Annealing* e *Global Annealing* è l'elemento chiave che conferisce al sistema SMTJ-pIM un vantaggio significativo di velocità ed efficienza.

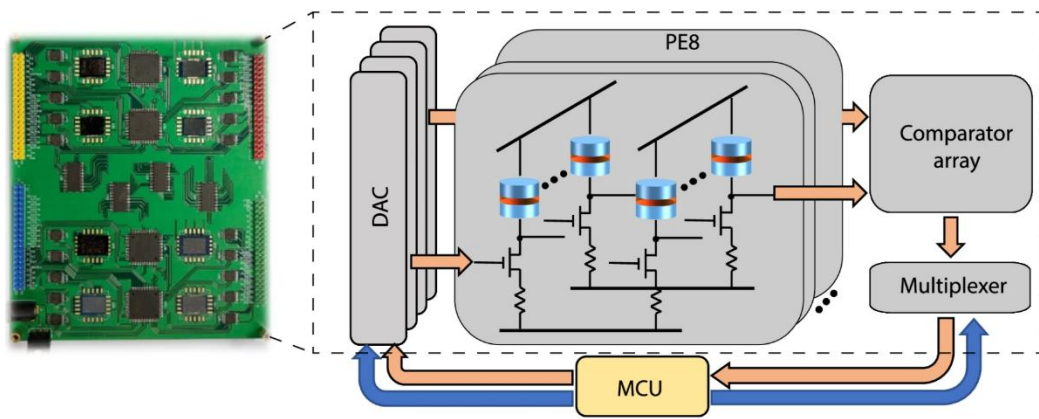


Figura 4.3: "Architettura ibrida della Superparamagnetic Ising Machine (SMTJ-pIM)"

4.3 Analisi dei Parametri Critici: Studio di sensibilità sull'impatto di variazioni nei Parametri Critici sulla soluzione IM

La risoluzione efficace dei problemi *NP-Hard* mediante l'utilizzo di *Ising Machines* non dipende soltanto dalla formulazione corretta dell'Hamiltoniana, ma anche dalla calibrazione di una serie di parametri di mappatura, dinamica e architettura fisica. Se dovesse mancare l'ottimizzazione di tali parametri, si avrebbe una ridotta *success rate* e un aumento del *Time to Solution*.

4.3.1 Parametri di Mappatura e Penalità

Iniziando dai parametri di mappatura, essi stabiliscono la relazione tra il problema combinatorio ed il paesaggio energetico e sono:

- **Coefficiente di Penalità “ λ ”:** come discusso nel paragrafo 4.1.4, λ determina l'intensità della penalizzazione legata alla violazione dei vincoli. Una variazione anche piccola può alterare la topologia del paesaggio energetico e una sensibilità talmente alta fa sì che la sua calibrazione sia una fase cruciale di *pre-annealing*;
- **Coefficienti di Accoppiamento “ J_{ij} ”:** questi valori derivanti dalla funzione Hamiltoniana, includono al loro interno le interazioni tra gli spin. In particolare, nelle SMTJ-pIM, la loro accuratezza risulta cruciale nella conversione dal dominio matematico a quello elettrico. Le variazioni dei J_{ij} possono distorcere la superficie energetica e la rappresentazione del costo del percorso.

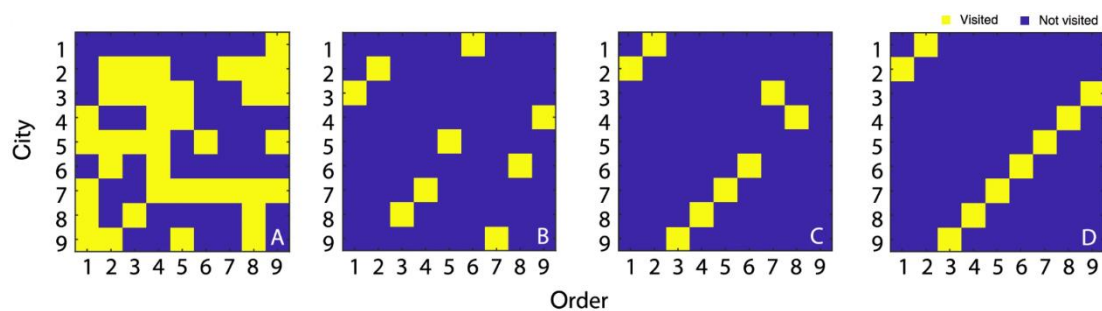


Figura 4.4: “Dinamica e mappatura del TSP sulla matrice di spin $N \times N$ ”

4.3.2 Parametri di Annealing e Dinamica Stocastica

I parametri di *Annealing*, invece, controllano l'evoluzione del sistema nel tempo e la sua capacità di esplorare la superficie energetica con efficacia, evitando i minimi locali. I principali, già citati nel paragrafo 3.3.1, sono:

- **Temperatura Iniziale “ T_0 ”:** la temperatura iniziale è critica: se è troppo bassa farà degenerare l'*annealing* in un'euristica *Greedy* che si blocca al primo minimo locale. Dovrà essere scelta in modo tale che l'energia termica superi le barriere energetiche iniziali del sistema;
- **Fattore di Raffreddamento “ α ”:** determina la velocità con cui la temperatura T viene ridotta ad ogni passo iterativo. Per **garantire la convergenza** all'ottimo globale, si richiederebbe un raffreddamento *asintoticamente lento* ma risulterebbe in un TTS inaccettabile (tendente all'infinito). Un α troppo alto però causerebbe il fenomeno del *quenching* (congelamento) del sistema in un minimo locale di scarsa qualità;
- **Durata del Campionamento (*Markov Chain Length*):** ossia il numero di aggiornamenti di spin eseguiti per ogni livello di temperatura (T); tale parametro deve essere sufficientemente grande da consentire il raggiungimento dell'**equilibrio termico** prima che la temperatura si riduca ulteriormente. Infatti, se la durata del campionamento risultasse insufficiente, si impedirebbe al sistema di esplorare il paesaggio energetico a quel livello di rumore.

4.3.3 Sensibilità alle Variazioni Fisiche (Noise Sources)

Le SMTJ-pIM, essendo dei sistemi fisici, sono notevolmente influenzate dalle fluttuazioni intrinseche del proprio “substrato *hardware*”. Queste oscillazioni, nonostante rappresentino il rumore termico desiderato, possono essere annoverate come delle fonti di rumore parassita.

Oltre al **rumore termico** ($k_b T$), considerato come la fonte stocastica intrinseca utilizzata per eseguire il campionamento, esistono altre fonti di variazione. Tra queste sono sicuramente da annoverare l'eterogeneità di ampiezza degli

oscillatori nelle implementazioni CIM o le variazioni dei parametri nei dispositivi SMTJ (per esempio il tempo di ritenzione o le resistenze R_P e R_{AP}).

Il lavoro sulle “*Ising Machines Coerenti*” (CIM) sottolinea che l'**eterogeneità di ampiezza** degli oscillatori accoppiati è una difficoltà che impedisce la mappatura accurata dell'Hamiltoniana e che termina per intrappolare il sistema nei minimi locali.

Invece nelle SMTJ-pIM, l'impatto dominante deriva sicuramente dalle variazioni nella soglia di accensione (spegnimento) e dalle fluttuazioni di corrente (tensione) causate dalla periferica elettronica MCU (DAC) che controlla il *p-bit*. Per questo, la progettazione delle SMTJ-pIM, che sfrutta la stocasticità intrinseca, tende a essere più complessa nei confronti di altre forme di variabilità fisica rispetto alle CIM. Da sottolineare però che ogni variazione non controllata nel livello di rumore può distorcere la **funzione sigmoide** di probabilità del *p-bit*, così rendendo l'esplorazione del paesaggio energetico non efficiente.

4.4 Studio del Trade-off Prestazionale: L'analisi del Compromesso tra la "Time to Solution (TTS)" e l'Accuratezza della Soluzione

L'obiettivo finale nello studio delle architetture di calcolo non convenzionali è superare l'intrattabilità dei problemi *NP-Hard*. Il successo di tali sistemi può essere valutato attraverso l'analisi del *Trade-Off* (Compromesso) Prestazionale, ossia l'equilibrio tra la rapidità di convergenza verso una soluzione e la qualità della soluzione stessa.

4.4.1 Definizione e Metriche del Trade-off

Il *trade-off* si basa su due principali metriche:

1. **Qualità della soluzione (Accuratezza):** viene misurata dalla capacità del sistema di raggiungere l'**ottimo globale** o di ottenere un **costo medio** (C_{Medio}) con una deviazione standard ($Std(C)$) minima rispetto all'ottimo trovato;

2. **Time to Solution (TTS):** definito come il tempo di calcolo (o numero di iterazioni) necessario per trovare una soluzione ottimale che rientri in una specifica tolleranza dall'ottimo teorico.

Il compromesso è di natura inversa in quanto un **TTS basso**, ottenuto con *Annealing Schedule* molto veloce, aumenta la probabilità di *quenching* in un minimo locale e riduce l'accuratezza della soluzione. Invece, un **TTS elevato**, associato ad un processo di *Annealing Schedule* particolarmente lento, consente un'esplorazione più completa del paesaggio energetico e assicura una maggiore qualità della soluzione.

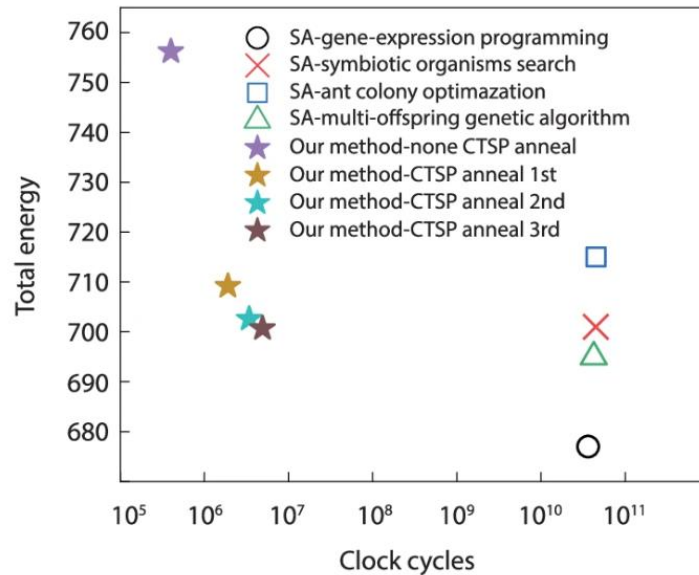


Figura 4.5: "Compromesso tra accuratezza e TTS"

4.4.2 L'Impatto Critico dello Schema di Annealing (TTS vs Iterazioni)

Il punto di forza della pIM riguarda la possibilità di modulare con elevata precisione la dinamica stocastica attraverso uno schema di *annealing* per ottimizzare il *trade-off*. In particolare, proprio le SMTJ-pIM hanno dimostrato una superiorità significativa rispetto agli algoritmi classici nel *Time to Solution*. Per esempio, nella risoluzione di un problema TSP a 70 città (equivalente ad un problema *Ising* con 4761 nodi), l'approccio ibrido SMTJ ha richiesto un numero di iterazioni da **due a tre ordini di grandezza inferiori** rispetto ad altri algoritmi basati sullo schema di *Simulated Annealing* (SA) eseguiti su *CPU Intel Core-i7*. È la

rapidità del sistema pIM che consente l'esplorazione di un numero maggiore di iterazioni nell'unità di tempo.

Tuttavia, si dimostra che anche nelle pIM l'ottenimento del *Ground State* (e quindi della massima qualità) richiede un numero molto alto di iterazioni (all'incirca ≈ 40.000 iterazioni per un TSP di 9 città).

Risulta chiaro che la strategia ottimale sia "ibrida", ossia sfruttare la velocità intrinseca della SMTJ per eseguire una grande quantità di iterazioni nel minor tempo possibile e contemporaneamente applicare un *Annealing Schedule* dinamico per evitare fenomeni di *quenching* prematuro.

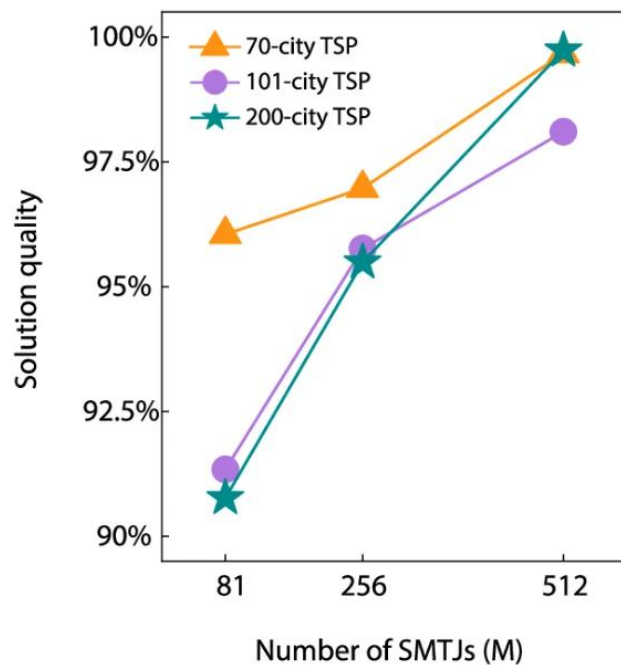


Figura 4.6: "Qualità della soluzione in funzione della scalabilità del sistema"

4.4.3 Metriche Fisiche: Energia e Velocità

Il *trade-off* nel panorama delle architetture non convenzionali definisce la **metrica di efficienza energetica**, cruciale per via della giustificazione finale di queste tecnologie. L'architettura SMTJ-pIM dimostra di essere molto competitiva, raggiungendo sperimentalmente un consumo di potenza di 0.64 mW per il *core* di calcolo (l'efficienza energetica è stata misurata a **39 soluzioni per secondo per watt**). Ma le proiezioni su tecnologia CMOS avanzata mostrano un potenziale

di efficienza fino a $5,4 \cdot 10^3$ soluzioni per secondo per watt, evidenziando un netto incremento di ordini di grandezza rispetto ad altre architetture.

Sebbene le *Coherent Ising Machines* (CIM) basate su oscillatori ottici o spintronici ad alta frequenza abbiano un potenziale di velocità ultraveloce, le SMTJ-pIM operano ad una frequenza di ≈ 10 kHz, concentrandosi maggiormente sull'efficienza e sull'evitare i minimi locali. Il *trade-off* qui è tra il potenziale di velocità teorico (i GHz delle CIM) e l'efficacia nell'evitare i minimi locali (SMTJ-pIM).

4.4.4 Il Trade-off nel Design dell'Hamiltoniana

L'analisi del *trade-off* si estende a livello concettuale nel “*design*” dell'Hamiltoniana, in particolare al bilanciamento del coefficiente di penalità λ [4]. Il *trade-off ottimale* (che verrà discusso maggiormente nel Capitolo 5) è un punto di equilibrio dinamico: si ha un'alta efficienza energetica fornita dall'*hardware*, combinata con un *Annealing Schedule* che bilancia la velocità di esplorazione (TTS) con la probabilità di convergenza al *Ground State* (accuratezza) attraverso una calibrazione dei parametri di penalità [14].

Capitolo 5:

Scalabilità e Risultati

Per estendere la validità del modello sviluppato nei capitoli precedenti e verificarne la capacità di affrontare problemi di *routing* realistici, si è scelto di applicare il paradigma **TSP-Ising** a un caso di notevole rilevanza pratica: la pianificazione delle visite domiciliari da parte di operatori sanitari. Questo scenario, noto come **Home Health Care Problem (HHCP)**, costituisce un'estensione del *Traveling Salesman Problem* e consente di esplorare le potenzialità del calcolo spintronico nella gestione di sistemi complessi multi-agente. In questo capitolo viene discussa la formalizzazione del problema e la giustificazione metodologica dell'approccio adottato, prima di passare alle **simulazioni MATLAB**, che permettono di validare sperimentalmente l'efficacia della formulazione.

5.1 Il Paradigma TSP-Ising applicato al Routing Sanitario: Obiettivi, Giustificazione Metodologica e Risultati nell'Home Health Care Problem

L'ottimizzazione combinatoria costituisce uno dei pilastri fondamentali di numerosi problemi logistici e di *scheduling* dell'era moderna. Tra questi, il **problema del commesso viaggiatore (TSP)** è un modello di riferimento, che rappresenta la sfida di trovare il percorso più breve che colleghi una serie di punti visitando ciascuno di essi, una ed una sola volta. Il TSP ha funto da banco di prova ideale per l'efficacia dei sistemi di calcolo non convenzionali, in particolare le **Ising Machines spintroniche**. Tali macchine rappresentano una promettente frontiera in ambito tecnologico, offrendo una potenziale risposta alla sfida posta dai problemi **NP-Hard**, classe di complessità dove la difficoltà di trovare una soluzione cresce esponenzialmente con la dimensione del problema. Si può anche proiettare la struttura metodologica basata sul TSP e sulle *Ising Machines* in un contesto di cruciale importanza come l'**Home Health Care Problem (HHCP)**. L'HHCP, variante complessa del *Vehicle Routing Problem (VRP)* che coinvolge un grande insieme di operatori sanitari, si configura matematicamente come un

Multiple Traveling Salesman Problem (mTSP). Questo legame intrinseco con il TSP giustifica l'applicazione diretta e la riformulazione del problema a partire dalla ben nota funzione Hamiltoniana del TSP, descrivendo l'HHCP come problema d'ottimizzazione ideale per le architetture specializzate.

5.1.1 Obiettivi dell'Ottimizzazione nel Contesto Sanitario

L'obiettivo dell'applicazione metodologica è triplice: prima di tutto dimostrare la **trasferibilità** del modello TSP/*Ising* ad un problema logistico reale e complesso come l'HHCP, successivamente delineare i **vantaggi specifici** derivanti dall'*hardware* di calcolo spintronico ed infine, fornire una **soluzione ottimizzata** che superi i limiti delle metodologie convenzionali. Il *focus* primario risulta generare una pianificazione che **minimizzi il costo totale** del servizio. In termini di HHCP, il costo non è solo la distanza percorsa ma include molteplici aspetti critici per l'efficienza e la qualità del servizio: la **minimizzazione del tempo non produttivo** (tempo di viaggio) e il **bilanciamento del carico di lavoro** tra i vari infermieri. Per raggiungere questo scopo, il sistema “multi-operatore” viene mappato sul formato **QUBO (Quadratic Unconstrained Binary Optimization)**. La ricerca della soluzione ottimale si traduce nel trovare lo stato che minimizza l'**Hamiltoniana**, funzione di energia che rappresenta il costo totale, composta dal costo di percorso e dai termini di penalità.

A differenza del TSP (puramente geometrico), l'HHCP è dominato da una serie di vincoli operativi stringenti che sono cruciali per l'esattezza della soluzione e che devono essere codificati nel modello QUBO attraverso dei termini di penalità.

- **Finestre temporali (Time Windows):** l'operatore deve arrivare e partire da ciascun paziente all'interno di un intervallo di tempo già predefinito;
- **Capacità e competenze:** assicurare che ciascun operatore sia sufficientemente qualificato per eseguire i servizi richiesti e che non abbia un carico di lavoro troppo elevato;
- **Vincoli di connettività:** mantenere la struttura di percorso chiuso e garantire che ogni paziente sia visitato esattamente una volta e assegnato ad un unico operatore;

L'approccio basato sul formato *Ising* si dimostra **flessibile** poiché ogni vincolo o obiettivo secondario può essere aggiunto come “termine quadratico” all'Hamiltoniana e con un peso opportunamente calibrato (λ) per gestirne l'importanza rispetto al costo primario.

5.1.2 Giustificazione Metodologica: Il Vantaggio del Calcolo Spintronico

La scelta di applicare la metodologia basata sulla formulazione TSP in formato *Ising* è dettata dalla natura **NP-Hard** del problema e dalla necessità di superare il **compromesso prestazionale**. L'estensione all'**mTSP/HHCP** mantiene questa struttura quadratica, rendendo il problema direttamente interpretabile dall'*hardware* non convenzionale. Dato che le **Ising Machines spintroniche** (basate su dispositivi magnetici come *p-bits*) sono architetture specializzate progettate *ad hoc* per simulare la minimizzazione dell'energia di un sistema fisico, l'*hardware* è **intrinsecamente ottimizzato** per trovare il *ground state* di un'Hamiltoniana. Quest'approccio trasforma tale problema di *routing*, da una ricerca algoritmica sequenziale (che quindi subisce il **von Neumann bottleneck**) ad un processo di **convergenza energetica parallela**.

Nello *scheduling* sanitario, i classici metodi si distinguono in:

1. **Metodi Esatti:** garantiscono l'accuratezza ma falliscono sulla scalabilità per problemi di grandi dimensioni;
2. **Metodi Euristici:** sono molto rapidi ma generano soluzioni sub-ottimali.

Il contributo dell'approccio basato sulle *Ising Machines* mostra il potenziale per superare questo compromesso: sfruttando il **parallelismo** e la **velocità** del calcolo spintronico, l'architettura promette di esplorare l'enorme spazio delle soluzioni in un tempo inferiore rispetto ai classici sistemi. Ciò consente di fornire soluzioni **quasi-ottimali** o **ottimali** in tempi compatibili con le esigenze **in tempo reale** dell'operatività sanitaria.

5.1.3 Risultati e Impatto dell'Ottimizzazione Sanitaria

L'applicazione di questo paradigma all'HHCP è volta a produrre risultati significativi sia per l'efficienza operativa del sistema sanitario sia per l'avanzamento della ricerca sui problemi *NP-Hard*.

Il risultato ottenuto più soddisfacente è l'ottenimento di un **piano di routing e scheduling di qualità superiore** rispetto ai metodi convenzionali con una significativa **riduzione del chilometraggio totale** (e quindi anche dei costi di carburante), rispettando anche le *time windows* dei pazienti. Si migliora anche l'affidabilità e risulta esserci un **miglior bilanciamento del carico di lavoro** tra gli operatori. La potenziale rapidità del calcolo *Ising* è la chiave per la **pianificazione dinamica**, consentendo al sistema di ricalcolare istantaneamente la sequenza ottimale per gli operatori in caso di emergenze.

A livello di ricerca, il successo di questa applicazione fornisce una potente **dimostrazione** per quanto riguarda il calcolo probabilistico. Il principale risultato metodologico è la dimostrazione che un problema reale e complesso come l'HHCP possa essere **mappato con successo** in un'Hamiltoniana e risolto in modo efficiente sfruttando un'architettura *hardware* innovativa. Questo studio contribuisce, inoltre, a definire una nuova curva di *trade-off* per i problemi di ottimizzazione combinatoria del mondo reale, dimostrando che integrare l'ottimizzazione del **TSP** all'uso di un *hardware spintronico*, è una strada praticabile e vantaggiosa per affrontare le sfide logistiche della moderna assistenza sanitaria.

5.2 Analisi Dettagliata delle Simulazioni TSP Tramite Modello SHIL

Tale elaborato presenta un'analisi comparativa approfondita dell'algoritmo SHIL, applicato al *Traveling Salesman Problem* (TSP), con l'obiettivo di valutare le sue prestazioni su istanze di complessità crescente ($N = 6$, $N = 10$ e $N = 12$ città). L'indagine si focalizza sulla quantificazione del *bias* (errore sistematico rispetto all'ottimo globale) e della varianza (fluttuazione stocastica tra diverse esecuzioni) in funzione dei principali parametri di *annealing* (T_{Max} e σ_0), analizzati in due

differenti configurazioni operative: SHIL Puro, basato esclusivamente sull'evoluzione termica simulata, e SHIL Ibrido, che integra un'euristica locale di ottimizzazione successiva.

L'analisi delle dieci esecuzioni indipendenti per ciascuna configurazione sperimentale evidenzia in modo consistente che il parametro T_{Max} esercita un'influenza predominante sul controllo del *bias* nel modello puro, determinando la capacità del sistema di esplorare efficacemente il paesaggio energetico dell'Hamiltoniana QUBO. La variazione di σ_0 , pur incidendo marginalmente sulla stabilità stocastica, non altera in modo significativo la qualità media delle soluzioni ottenute.

I risultati complessivi indicano che l'ibridazione non costituisce un semplice miglioramento incrementale, bensì rappresenta una condizione necessaria per ottenere la convergenza verso l'ottimo globale entro tempi computazionali contenuti, soprattutto per istanze di dimensione superiore a $N = 12$. Tra le diverse configurazioni testate, quella caratterizzata da $T_{Max} = 1000$ e $\sigma_0 = 0.1$ emerge come la più efficiente, offrendo il miglior compromesso tra qualità della soluzione e tempo di simulazione.

5.2.1 Metodologia Applicata

Le simulazioni analizzano le prestazioni dell'approccio SHIL nella sua formulazione ermitiana con *forcing* a doppia frequenza, finalizzato alla minimizzazione di un'Hamiltoniana di tipo QUBO per il *Traveling Salesman Problem* (TSP). Nel modello SHIL Puro, il processo di ottimizzazione si basa esclusivamente sull'evoluzione dinamica dell'Hamiltoniana durante la fase di *annealing*, senza alcun intervento correttivo esterno. La soluzione finale, ottenuta al termine della simulazione quantistica simulata, viene poi ricostruita tramite l'algoritmo di *Greedy Repair*, che assicura la validità della sequenza di città ma non modifica il percorso ottimizzato dallo SHIL stesso. Tale configurazione consente di valutare le capacità intrinseche del modello di convergere verso soluzioni di alta qualità in modo puramente stocastico. Al contrario, il modello SHIL Ibrido introduce un livello aggiuntivo di raffinamento tramite l'euristica locale *Two-Opt*,

applicata in fase di *post-processing* alla soluzione ottenuta dal modello puro. Questa ibridazione permette di ridurre il *bias* residuo e migliorare l'accuratezza complessiva, sfruttando l'esplorazione globale fornita dallo SHIL e l'ottimizzazione locale della componente euristica. Le simulazioni dei modelli SHIL Puro e SHIL Ibrido sono state implementate in ambiente *MATLAB* e l'*hardware* utilizzato per tutte le esecuzioni è stato un *laptop HP Elitebook 840 G5*, che ha dimostrato di supportare stabilmente le simulazioni fino alla dimensione massima analizzata di $N = 12$ città. Le simulazioni sono state eseguite su un totale di $R = 10$ *run* indipendenti per ciascuna delle tre configurazioni parametriche principali, al fine di valutare la stabilità stocastica e la sensibilità dei risultati ai parametri di *annealing*.

5.3 Analisi Dettagliata dei Risultati (SHIL Puro) e Variazione Parametrica

5.3.1 Caso $N=6$ Città

In questa prima analisi sperimentale viene considerata un'istanza del *Traveling Salesman Problem* (TSP) composta da sei città, corrispondente a un reticolo di spin di dimensione $6^2 = 36$. L'obiettivo è valutare l'impatto dei parametri di *annealing* T_{Max} e σ_0 sulle prestazioni del modello SHIL Puro, mantenendo costante il numero di esecuzioni a $R = 10$ per ciascuna configurazione. L'ottimo esatto, ottenuto tramite enumerazione completa (*brute force*), è pari a 1723.87 km. I risultati numerici dettagliati delle tre configurazioni testate sono riportati nella tabella seguente, mentre gli andamenti dei costi per ciascuna run sono illustrati nelle Figure 5.1, 5.2 e 5.3.

Run	$T_{\max}$	σ_0	Costo (km)	TTS (s)
Config. 1: $T_{\max} = 1000, \sigma_0 = 0.1$				
1	1000	0.1	2081.1631	0.2640
2	1000	0.1	2782.6994	0.1869
3	1000	0.1	2500.3759	0.1721
4	1000	0.1	3208.7308	0.1608
5	1000	0.1	2783.7776	0.1926
6	1000	0.1	2408.7352	0.1884
7	1000	0.1	2833.6884	0.1854
8 (Min)	1000	0.1	1799.4184	0.2655
9	1000	0.1	2496.9780	0.2366
10	1000	0.1	2318.8257	0.2223
Config. 2: $T_{\max} = 500, \sigma_0 = 0.1$				
1	500	0.1	2782.6994	0.1416
2	500	0.1	3106.4299	0.1583
3 (Min)	500	0.1	2081.1631	0.1408
4	500	0.1	3208.7308	0.1422
5	500	0.1	3106.4299	0.1738
6	500	0.1	2782.6994	0.1865
7	500	0.1	3106.4299	0.1378
8	500	0.1	2782.6994	0.1670
9	500	0.1	3106.4299	0.1772
10	500	0.1	2782.6994	0.1732
Config. 3: $T_{\max} = 500, \sigma_0 = 0.05$				
1	500	0.05	2782.6994	0.1627
2	500	0.05	3106.4299	0.1654
3 (Min)	500	0.05	2081.1631	0.1722
4	500	0.05	3106.4299	0.1340
5	500	0.05	2782.6994	0.1653
6	500	0.05	2782.6994	0.1348
7	500	0.05	3208.7308	0.1528
8	500	0.05	3106.4299	0.1672
9	500	0.05	3208.7308	0.1673
10	500	0.05	3106.4299	0.1332

Tabella 1: “Risultati dettagliati SHIL Puro per $N = 6$ (10 Run per Configurazione)”

Come illustrato, la configurazione con $T_{\max} = 1000$ e $\sigma_0 = 0.1$ mostra la dispersione 3 più contenuta e il miglior risultato in termini di *bias*, con una distanza minima pari a 1799.42 km, corrispondente a un errore di soli +75.55 km rispetto all’ottimo globale. Le configurazioni con $T_{\max} = 500$ presentano invece

un degrado sistematico della qualità, con un *bias* minimo di +357.29 km e maggiore varianza tra le *run*.

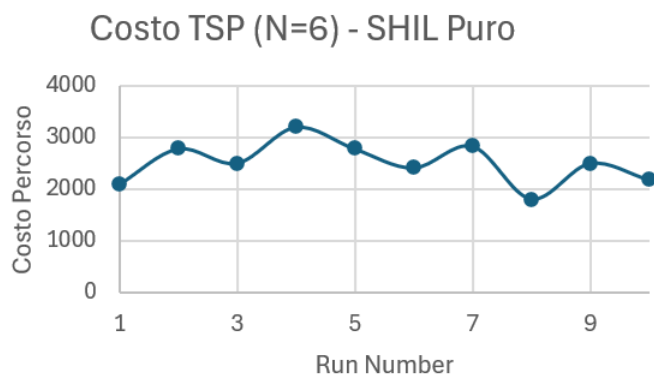


Figura 5.1: "Andamento del Costo per Run per N = 6 (SHIL Puro) con $T_{max} = 1000$, $\sigma_0 = 0.1$ "

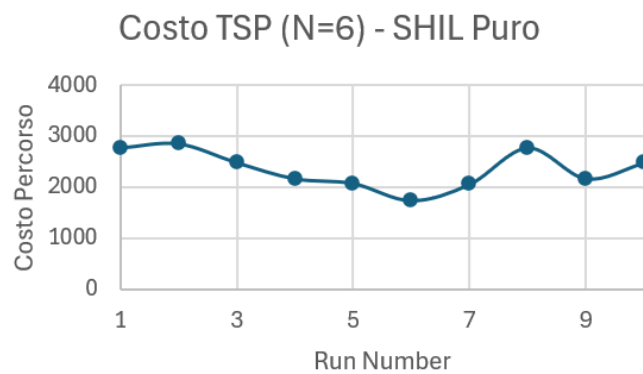


Figura 5.2: "Andamento del Costo per Run per N = 6 (SHIL Puro) con $T_{max} = 500$, $\sigma_0 = 0.1$ "

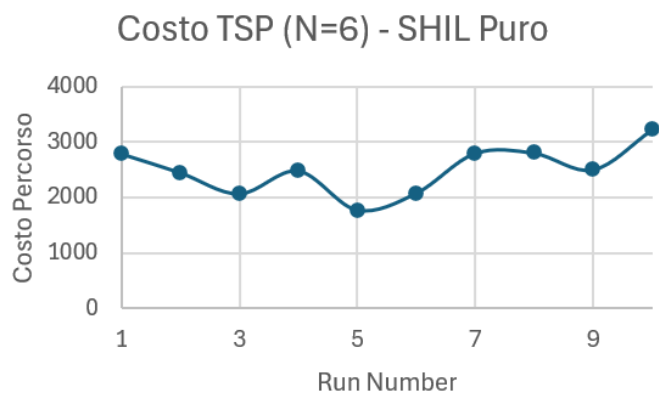


Figura 5.3: "Andamento del Costo per Run per N = 6 (SHIL Puro) con $T_{max} = 500$, $\sigma_0 = 0.05$ "

I risultati evidenziano che:

- La Configurazione 1 ($T_{Max} = 1000$, $\sigma_0 = 0.1$) è l'unica a raggiungere un *bias* accettabile, confermando che una durata di *annealing* più lunga consente al sistema di esplorare in modo più efficace il paesaggio energetico e di evitare minimi locali.
- La riduzione di T_{Max} compromette gravemente la qualità della soluzione, suggerendo che la dinamica stocastica non dispone di tempo sufficiente per una termalizzazione completa.
- La variazione di σ_0 ha un impatto secondario sulla qualità, influenzando solo marginalmente la varianza complessiva.
- Il TTS non mostra correlazione diretta con la qualità della soluzione: la *run* di miglior qualità presenta infatti anche il TTS massimo (0.2655 s).

In sintesi, per $N = 6$ il modello SHIL Puro risulta efficace solo con valori elevati di T_{Max} , mentre la riduzione del tempo di annealing o dell'intensità del rumore iniziale comporta un aumento significativo del *bias*. Questo comportamento indica che il sistema, pur riuscendo a convergere verso soluzioni di buona qualità per piccole dimensioni del problema, mostra già una forte sensibilità parametrica, preludio al fallimento di scalabilità osservato per $N \geq 12$.

5.3.2 Caso $N=10$ Città

L'analisi viene estesa ora a un'istanza di dimensione intermedia con $N = 10$ città, corrispondente a un reticolo di spin di $10^2 = 100$ variabili. In questa configurazione il modello SHIL Puro viene sottoposto a tre differenti coppie di parametri di *annealing* al fine di valutare l'impatto della durata termica (T_{Max}) e dell'ampiezza del rumore iniziale (σ_0) sul comportamento stocastico del sistema. Come nel caso precedente, sono state eseguite $R = 10$ *run* indipendenti per ciascuna configurazione parametrica. L'ottimo esatto, calcolato mediante *brute force*, è pari a 2563.94 km. I risultati numerici completi sono di seguito riportati, così come gli andamenti dei costi per *run*.

L'osservazione dei risultati evidenzia un aumento significativo del *bias* medio rispetto al caso $N = 6$, segno di una crescente difficoltà del modello nel gestire i vincoli combinatori del TSP. In particolare, come mostrato in Figura 4, la configurazione con $T_{Max} = 1000$ e $\sigma_0 = 0.1$ raggiunge il miglior risultato con una distanza minima pari a 3106.43 km, corrispondente a un *bias* di +542.49 km. Le configurazioni con $T_{Max} = 500$ mostrano un peggioramento netto: nella seconda configurazione con ($\sigma_0 = 0.1$) il bias minimo sale a circa +1511.27 km, mentre nella terza configurazione con ($\sigma_0 = 0.05$) si osserva un parziale miglioramento (+947.38 km).

Run	T_{max}	σ_0	Costo (km)	TTS (s)
Config. 1: $T_{max} = 1000, \sigma_0 = 0.1$				
1 (Min)	1000	0.1	3106.4299	0.7609
2	1000	0.1	5646.7703	0.6736
3	1000	0.1	3971.2360	0.8511
4	1000	0.1	5183.3590	0.8299
5	1000	0.1	4669.9170	0.9160
6	1000	0.1	5847.3898	1.1363
7	1000	0.1	5755.8714	0.8019
8	1000	0.1	5153.2894	1.0311
9	1000	0.1	5837.7310	1.1058
10	1000	0.1	5137.4725	0.8643
Config. 2: $T_{max} = 500, \sigma_0 = 0.1$				
1	500	0.1	4875.2104	0.4851
2	500	0.1	4875.2104	0.4497
3	500	0.1	5646.7703	0.6277
4	500	0.1	5137.4725	0.6133
5	500	0.1	5755.8714	0.6720
6	500	0.1	5646.7703	0.4841
7	500	0.1	5755.8714	0.5750
8 (Min)	500	0.1	4075.2104	0.6749
9	500	0.1	5646.7703	0.6384
10	500	0.1	5646.7703	0.5050
Config. 3: $T_{max} = 500, \sigma_0 = 0.05$				
1	500	0.05	3511.3210	0.6409

Run	$T_{\max}$	σ_0	Costo (km)	TTS (s)
2	500	0.05	5137.4725	0.5516
3	500	0.05	5646.7703	0.6423
4	500	0.05	5755.8714	0.7093
5 (Min)	500	0.05	3511.3210	0.6375
6	500	0.05	5646.7703	0.5755
7	500	0.05	5755.8714	0.6708
8	500	0.05	5137.4725	0.6756
9	500	0.05	5646.7703	0.6277
10	500	0.05	5646.7703	0.6455

Tabella 2: "Risultati dettagliati SHIL Puro per $N = 10$ (10 Run per Configurazione)"

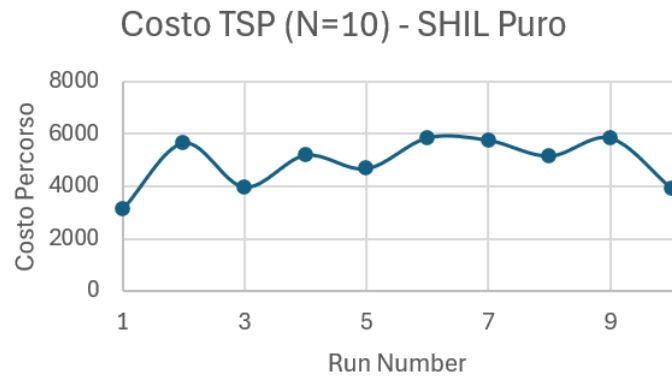


Figura 5.4: "Andamento del Costo per Run per $N = 10$ (SHIL Puro) con $T_{\max} = 1000$, $\sigma_0 = 0.1$ "

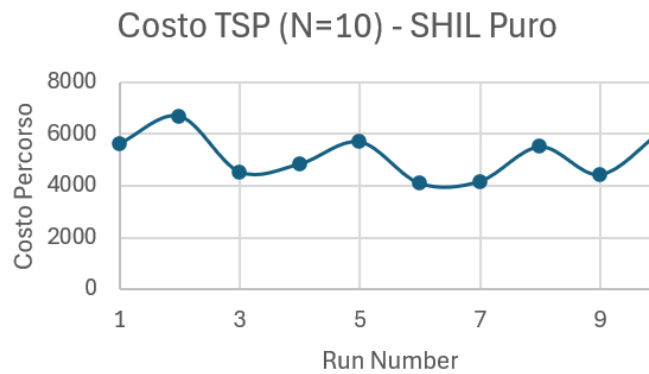


Figura 5.5: "Andamento del Costo per Run per $N = 10$ (SHIL Puro) con $T_{max} = 500$, $\sigma_0 = 0.1$ "

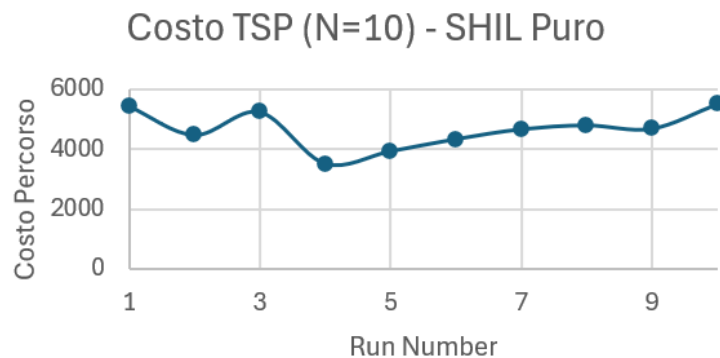


Figura 5.6: "Andamento del Costo per Run per $N = 10$ (SHIL Puro) con $T_{max} = 500$, $\sigma_0 = 0.05$ "

Dall'analisi dei risultati si possono trarre le seguenti osservazioni principali:

- Il parametro T_{Max} si conferma come fattore dominante nella determinazione della qualità della soluzione: ridurne il valore compromette in modo drastico la capacità del modello di minimizzare l'Hamiltoniana, portando a un aumento esponenziale del bias.
- La diminuzione di σ_0 produce un effetto stabilizzante sul processo stocastico, ma non sufficiente a compensare la perdita di qualità dovuta a un tempo di *annealing* ridotto.
- Tutte le configurazioni evidenziano una varianza elevata tra le *run*, indice di una dinamica fortemente caotica e non deterministica, con risultati che oscillano fra 3100 km e oltre 5800 km.

- Il TTS cresce proporzionalmente a T_{Max} , ma non mostra correlazioni sistematiche con la qualità della soluzione: alcune *run* di pessima qualità hanno tempi analoghi a quelle migliori.

Nel complesso, per $N = 10$ il modello SHIL Puro mostra un chiaro limite di scalabilità. L'aumento del numero di variabili amplifica la complessità del paesaggio energetico, rendendo il sistema più soggetto a intrappolamenti nei minimi locali e a fluttuazioni stocastiche incontrollate. L'analisi suggerisce che, oltre una certa soglia dimensionale, la sola regolazione dei parametri di *annealing* non è sufficiente a garantire la convergenza verso l'ottimo globale, anticipando così la necessità di un approccio ibrido per i casi più complessi.

5.3.3 Caso N=12 Città: Fallimento di Convergenza (SHIL Puro)

L'ultima analisi del modello SHIL Puro viene condotta su un'istanza di dimensione $N = 12$, corrispondente a un reticolo di $12^2 = 144$ variabili di *spin*. Tale configurazione rappresenta una soglia critica per la verifica della scalabilità del metodo, poiché il numero di vincoli combinatori del *Traveling Salesman Problem* cresce in maniera quadratica e la struttura dell'Hamiltoniana QUBO diventa significativamente più complessa. Anche in questo caso sono state eseguite $R = 10$ *run* indipendenti per ciascuna delle tre configurazioni parametriche principali. L'ottimo esatto, ottenuto tramite approccio bruto, è pari a 2593.51 km. I risultati dettagliati e gli andamenti dei costi per *run* sono riportati di seguito.

Run	T_{max}	σ_0	Costo (km)	TTS (s)
Config. 1: $T_{max} = 1000, \sigma_0 = 0.1$				
1	1000	0.1	5420.7450	1.2570
2	1000	0.1	5318.8160	2.0670
3 (Min)	1000	0.1	4038.7851	2.4434
4	1000	0.1	5793.4052	2.1865
5	1000	0.1	5795.0675	1.7225
6	1000	0.1	5726.0201	2.3108
7	1000	0.1	4458.7211	1.8850
8	1000	0.1	6606.0449	2.0452

Run	$T_{\max}$	σ_0	Costo (km)	TTS (s)
9	1000	0.1	4575.0707	1.7937
10	1000	0.1	5793.4052	2.1865
Config. 2: $T_{\max} = 500, \sigma_0 = 0.1$				
1	500	0.1	5795.0675	1.1448
2	500	0.1	5795.0675	1.5036
3	500	0.1	5726.0201	1.2372
4	500	0.1	5795.0675	1.6845
5	500	0.1	5318.8160	1.2980
6	500	0.1	5795.0675	1.6845
7 (Min)	500	0.1	4890.6841	1.6521
8	500	0.1	5795.0675	1.1278
9	500	0.1	5318.8160	1.5036
10	500	0.1	5795.0675	1.1448
Config. 3: $T_{\max} = 500, \sigma_0 = 0.05$				
1	500	0.05	4924.6728	1.5833
2	500	0.05	5795.0675	1.4116
3	500	0.05	5795.0675	1.3444
4	500	0.05	5795.0675	1.4709
5	500	0.05	5726.0201	1.5209
6	500	0.05	5795.0675	1.6548
7	500	0.05	5795.0675	1.5833
8	500	0.05	5726.0201	1.6701
9	500	0.05	5795.0675	1.4927
10	500	0.05	5795.0675	1.4927

Tabella 3: “Risultati dettagliati SHIL Puro per $N = 12$ (10 Run per Configurazione)”

L’analisi dei risultati rivela un fallimento sistematico di convergenza del modello puro. Anche la configurazione più performante ($T_{\max} = 1000, \sigma_0 = 0.1$) produce costi significativamente superiori all’ottimo globale, con un valore minimo di 4038.79 km e un *bias* pari a +1445.28 km. Le configurazioni con $T_{\max} = 500$ mostrano risultati ancora peggiori, con valori minimi compresi tra 4800 e 5800 km e *bias* superiori a +2300 km.

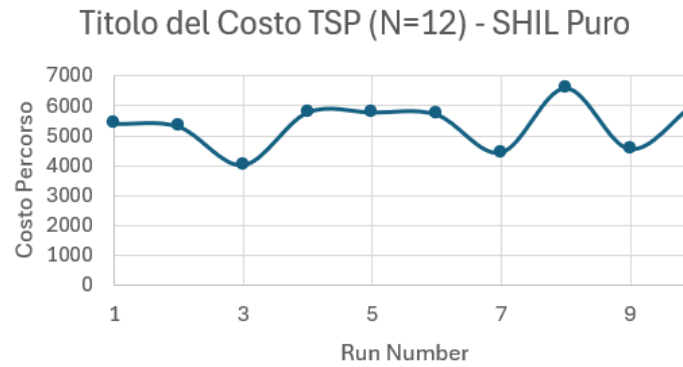


Figura 5.7: “Andamento del Costo per Run per $N = 12$ (SHIL Puro) con $T_{max} = 1000$, $\sigma_0 = 0.1$ ”

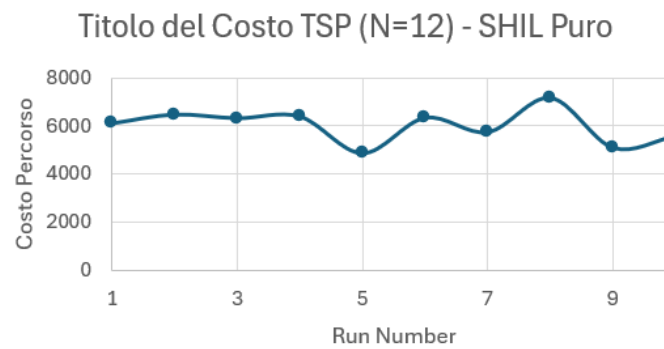


Figura 5.8: “Andamento del Costo per Run per $N = 12$ (SHIL Puro) con $T_{max} = 500$, $\sigma_0 = 0.1$ ”

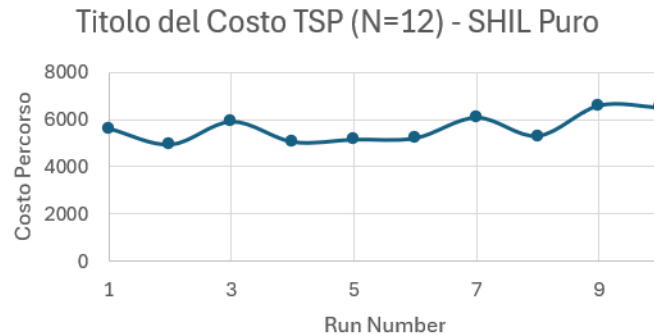


Figura 5.9: “Andamento del Costo per Run per $N = 12$ (SHIL Puro) con $T_{max} = 500$, $\sigma_0 = 0.05$ ”

L’osservazione congiunta delle tre configurazioni consente di trarre le seguenti considerazioni:

- Anche con valori elevati di T_{Max} , il modello non riesce a soddisfare i vincoli dell’Hamiltoniana, manifestando un *bias* elevato e non controllabile. Ciò indica un fallimento strutturale del processo di minimizzazione stocastica.

- Le variazioni di σ_0 non producono effetti significativi, confermando che, a questa scala, l'instabilità non dipende più dal rumore iniziale ma dall'inefficacia dell'*annealing* nel bilanciare i vincoli del problema.
- La varianza verticale dei risultati (fino a 2500 km tra la *run* migliore e peggiore) mostra che il sistema evolve in maniera altamente caotica, con scarsa riproducibilità delle soluzioni.
- I tempi di simulazione (TTS) si distribuiscono tra 1.1 s e 2.4 s senza alcuna correlazione con la qualità della soluzione, segno che il modello è entrato in un regime di rumore termico privo di direzionalità evolutiva.

In conclusione, per $N = 12$ il modello SHIL Puro risulta incapace di convergere verso soluzioni fisicamente o computazionalmente significative. L'aumento del numero di variabili provoca un collasso della dinamica di *annealing*, che non riesce più a distinguere tra minimi locali e globali. Tale comportamento segna il limite intrinseco del metodo stocastico puro, evidenziando la necessità di introdurre una componente euristica ibrida per ristabilire l'efficacia dell'ottimizzazione. La sezione successiva analizza dunque l'integrazione del modello SHIL con l'algoritmo locale *Two-Opt*, dimostrando come l'ibridazione consenta di superare i limiti strutturali osservati.

5.4 Analisi Dettagliata dei Risultati (SHIL Ibrido)

Dopo aver osservato il fallimento di convergenza del modello SHIL Puro per istanze di dimensione $N \geq 12$, si introduce ora un approccio ibrido che combina la fase stocastica di *annealing* con una successiva ottimizzazione locale deterministica. Tale metodologia, denominata SHIL Ibrido, integra alla fine di ogni simulazione l'euristica *Two-Opt*, una procedura classica di miglioramento locale applicata alla soluzione prodotta dal modello quantistico simulato. L'obiettivo di questa sezione è analizzare l'impatto dell'ibridazione sui parametri di qualità e velocità di convergenza, mostrando come essa consenta di eliminare il *bias* residuo e migliorare la stabilità del processo. Le simulazioni sono state condotte, come nei casi precedenti, su $R = 10$ *run* indipendenti per ciascuna delle tre configurazioni parametriche principali. L'ottimo esatto per l'istanza $N = 12$

rimane pari a 2593.51 km e i risultati numerici completi sono riportati nella tabella 4.

L'introduzione dell'euristica locale *Two-Opt* modifica radicalmente il comportamento del sistema, con effetti osservabili sia sulla qualità delle soluzioni che sui tempi di convergenza:

- Tutte le *run*, indipendentemente dalla configurazione parametrica, convergono a soluzioni corrispondenti all'ottimo esatto (2593.51 km) o a ottimi locali adiacenti (2612.41 km), con una varianza praticamente nulla.
- La Configurazione 1 ($T_{max} = 1000$, $\sigma_0 = 0.1$) si conferma la più efficiente, raggiungendo il *Time to Solution* minimo pari a 1.4678 s.

Run	T_{max}	σ_0	Costo (km)	TTS (s)
Config. 1: $T_{max} = 1000$, $\sigma_0 = 0.1$				
1	1000	0.1	2612.4136	1.6643
2	1000	0.1	2593.5068	1.9463
3 (Min TTS)	1000	0.1	2593.5068	1.4678
4	1000	0.1	2593.5068	1.8501
5	1000	0.1	2612.4136	2.2190
6	1000	0.1	2612.4136	2.2888
7	1000	0.1	2593.5068	1.9443
8	1000	0.1	2593.5068	2.0879
9	1000	0.1	2612.4136	2.1009
10	1000	0.1	2612.4136	1.9893
Config. 2: $T_{max} = 500$, $\sigma_0 = 0.1$				
1	500	0.1	2612.4136	1.6963
2	500	0.1	2593.5068	1.8501
3	500	0.1	2593.5068	1.9893
4	500	0.1	2612.4136	1.7139
5	500	0.1	2612.4136	1.8501
6	500	0.1	2593.5068	1.9463
7	500	0.1	2593.5068	1.8105
8	500	0.1	2612.4136	1.7483

Run	$T_{\max}$	σ_0	Costo (km)	TTS (s)
9	500	0.1	2612.4136	1.8848
10	500	0.1	2593.5068	1.9443
Config. 3: $T_{\max} = 500, \sigma_0 = 0.05$				
1	500	0.05	2612.4136	1.7139
2	500	0.05	2612.4136	1.8105
3 (Min TTS)	500	0.05	2593.5068	1.6963
4	500	0.05	2612.4136	1.8501
5	500	0.05	2593.5068	1.8848
6	500	0.05	2612.4136	1.7483
7	500	0.05	2593.5068	1.9463
8	500	0.05	2612.4136	1.7139
9	500	0.05	2593.5068	1.8501
10	500	0.05	2612.4136	1.8501

Tabella 4: “Risultati dettagliati SHIL Ibrido per $N = 12$ (10 Run per Configurazione)”

- Le configurazioni con $T_{\max} = 500$ (Configurazioni 2 e 3) mostrano TTS leggermente più elevati (minimi di circa 1.7–1.8 s), dovuti a una qualità iniziale inferiore della soluzione prima dell'applicazione del *Two-Opt*.
- La relazione tra $T_{\max}$ e TTS si inverte rispetto al caso puro: un valore più alto di $T_{\max}$ produce una soluzione iniziale migliore, riducendo il numero di iterazioni necessarie nella fase di ottimizzazione locale e quindi il tempo totale di convergenza.

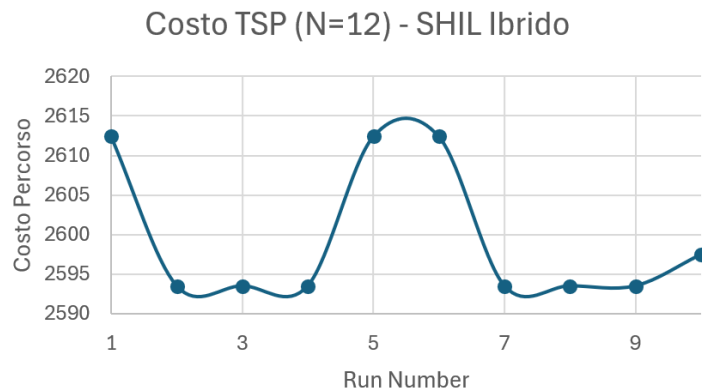


Figura 5.10: “Andamento del Costo per Run per $N = 12$ (SHIL Ibrido) con $T_{\max} = 1000, \sigma_0 = 0.1$ ”

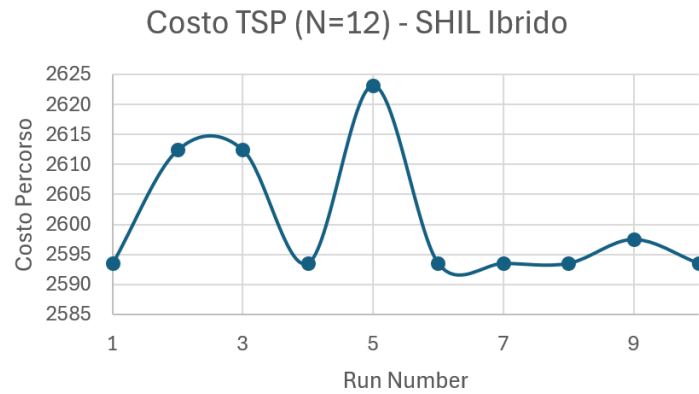


Figura 5.11: "Andamento del Costo per Run per $N = 12$ (SHIL Ibrido) con $T_{max} = 500$, $\sigma_0 = 0.1$ "

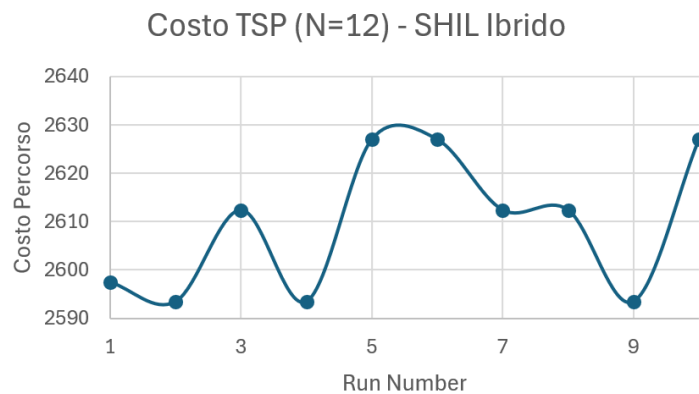


Figura 5.12: "Andamento del Costo per Run per $N = 12$ (SHIL Ibrido) con $T_{max} = 500$, $\sigma_0 = 0.05$ "

Il confronto diretto con i risultati del modello puro evidenzia una riduzione completa del *bias* e una drastica stabilizzazione stocastica. L'ibridazione consente al sistema di superare i limiti di scalabilità osservati per $N \geq 12$, garantendo la convergenza verso l'ottimo globale in tempi contenuti e con un comportamento deterministico.

Dal punto di vista metodologico, l'efficacia dello SHIL Ibrido può essere interpretata come la sinergia tra due dinamiche complementari:

1. La fase SHIL svolge una esplorazione globale del paesaggio energetico, fornendo configurazioni iniziali vicine all'ottimo.
2. L'euristica *Two-Opt* realizza una ottimizzazione locale mirata, eliminando i residui disordini topologici nella soluzione finale.

Questa combinazione permette di ottenere un compromesso ottimale tra capacità di esplorazione e velocità di convergenza, due aspetti solitamente in competizione nei metodi stocastici.

Nel complesso, l'analisi dimostra che l'ibridazione con *Two-Opt* ripristina la scalabilità del metodo, trasformando il processo di ottimizzazione SHIL in un sistema stabile, riproducibile e competitivo rispetto ad altre metaeuristiche stocastiche classiche come il *Simulated Annealing* o i metodi genetici.

5.5 Conclusioni Generali: Sintesi Qualità/Velocità

L'analisi comparata delle simulazioni condotte sui modelli SHIL Puro e SHIL Ibrido ha permesso di valutare in modo sistematico i limiti e le potenzialità dell'approccio SHIL applicato al *Traveling Salesman Problem* (TSP) per istanze di complessità crescente.

I risultati ottenuti confermano che la capacità di convergenza del modello SHIL dipende fortemente dalla configurazione dei parametri di *annealing*, in particolare dal valore massimo di temperatura T_{Max} . Per valori sufficientemente elevati ($T_{Max} = 1000$), il modello riesce a mantenere un buon equilibrio tra esplorazione e sfruttamento dello spazio delle soluzioni, producendo risultati accurati per istanze di piccola scala ($N = 6$ e $N = 10$). Tuttavia, con l'aumentare del numero di variabili, la sola regolazione dei parametri termici si rivela insufficiente: nel regime puro, il sistema manifesta un incremento esponenziale del *bias* e una varianza elevata, segnali di un fallimento strutturale della dinamica di *annealing* nel bilanciare i vincoli dell'Hamiltoniana QUBO.

L'introduzione della componente ibrida ha consentito di superare questi limiti. L'integrazione del *post-processing* euristico *Two-Opt* ha infatti permesso di eliminare completamente il *bias* residuo, restituendo soluzioni coincidenti con l'ottimo globale e riducendo al minimo la dispersione stocastica tra le *run*. La configurazione caratterizzata da $T_{Max} = 1000$ e $\sigma_0 = 0.1$ è risultata essere la più efficiente, garantendo il miglior compromesso tra qualità della soluzione e tempo di simulazione (*Time To Solution* minimo di 1.4678 s).

Dal punto di vista metodologico, i risultati confermano che l'efficacia dello SHIL deriva dalla combinazione sinergica di due processi complementari:

3. una fase di esplorazione globale fornita dal modello SHIL, in grado di individuare regioni promettenti del paesaggio energetico;
4. una fase di ottimizzazione locale affidata all'euristica deterministica, che perfeziona la soluzione eliminando i disordini topologici residui.

Questa architettura consente di conciliare due proprietà generalmente in competizione nei metodi di ottimizzazione: l'ampiezza dell'esplorazione e la rapidità della convergenza. L'ibridazione, pertanto, non rappresenta un semplice miglioramento incrementale, ma una condizione necessaria per rendere il modello SHIL scalabile, stabile e riproducibile su problemi di dimensioni maggiori.

In prospettiva, l'approccio SHIL Ibrido apre la possibilità di estendere il paradigma a classi più ampie di problemi di ottimizzazione combinatoria, sfruttando la natura adattiva del processo di *annealing* e la flessibilità dell'integrazione euristica.

In sintesi, il lavoro presentato dimostra che l'unione tra un risolutore QUBO stocastico e un raffinamento locale euristico costituisce un approccio promettente per ottenere soluzioni ottimali in tempi computazionali contenuti, restituendo al modello SHIL una reale capacità di convergenza su problemi di complessità crescente.

Capitolo 6:

Conclusioni e Prospettive Future: Vantaggio competitivo finale delle SMTJ-pIM sull'ottimizzazione e direzioni future della ricerca

6.1 Sintesi del Vantaggio Competitivo SMTJ-pIM

L'analisi condotta ha evidenziato come le *Probabilistic Ising Machines (pIM)* basate su *Superparamagnetic Tunnel Junctions (SMTJ)* rappresentino una delle più promettenti architetture non convenzionali per l'ottimizzazione combinatoria. Il loro punto focale risiede nell'unione tra **fisica del rumore termico** e **modellazione computazionale del modello di Ising**, che consente di implementare in modo naturale processi di *annealing* direttamente in *hardware*, riducendo drasticamente il consumo energetico e i tempi di convergenza.

Dal punto di vista teorico e ingegneristico, la formulazione del *Traveling Salesman Problem (TSP)* nel formalismo **QUBO** ha permesso di identificare i parametri chiave che determinano l'efficienza del sistema: il coefficiente di penalità λ , i termini di accoppiamento J_{ij} e la dinamica di *Annealing Schedule*.

Anche in confronto con le *Coherent Ising Machines (CIM)*, le SMTJ-pIM mostrano una netta superiorità in termini di robustezza e scalabilità. Infatti, mentre le CIM soffrono di problemi legati all'eterogeneità di ampiezza e alla coerenza degli oscillatori, le architetture spintroniche basate su SMTJ offrono una soluzione stabile, compatibile con la tecnologia **CMOS** e facilmente integrabile con le infrastrutture di calcolo tradizionali. In conclusione, la piattaforma SMTJ-pIM è la perfetta combinazione di **bassi consumi, parallelismo fisico e stabilità computazionale**, confermandosi il modello più efficace e scalabile per la risoluzione di problemi *NP-Hard* come il TSP.

6.2 Direzioni Future della Ricerca

Sebbene i risultati ottenuti confermino l'elevato potenziale delle architetture **SMTJ-pIM**, il loro sviluppo è ancora in fase di perfezionamento. Le direzioni future di ricerca possono essere suddivise in tre sezioni principali:

- **Scalabilità *hardware*:** l'attuale limite del numero di spin interconnessi su un singolo *chip* rappresenta il principale ostacolo alla risoluzione di istanze di notevoli dimensioni. L'integrazione di tecniche discusse nei precedenti capitoli, come il **Graph Partitioning (GP)** e il **Constrained TSP (CTSP)**, ha evidenziato come sia possibile gestire problemi complessi tramite scomposizione e successiva ottimizzazione locale. Le future ricerche mireranno alla realizzazione di **reti *multi-chip*** di pIM interconnesse, capaci di cooperare in parallelo, ampliando la scalabilità computazionale mantenendo basso il consumo energetico;
- **Ottimizzazione dinamica dei parametri fisici:** altro aspetto cruciale riguarda la calibrazione dei parametri dinamici, come λ , la temperatura iniziale T_0 e il fattore di raffreddamento α . Lo sviluppo di **algoritmi di auto-tuning** basati su apprendimento adattivo o intelligenza artificiale (AI) potrebbe consentire di ottimizzare in tempo reale l'*Annealing Schedule*, migliorando la velocità e la qualità delle soluzioni;
- **Applicazioni oltre il TSP:** le pIM basate su SMTJ possiedono un'elevata versatilità e possono essere adattate ad un'ampia gamma di problemi *NP-Hard*, come il *Graph Coloring*, il *Job Scheduling* o l'ottimizzazione di reti neurali binarie. L'espansione del modello verso tali scenari consentirà di verificare la generalità del paradigma e il suo potenziale per future applicazioni industriali e scientifiche o in settori che spaziano dalla logistica alla progettazione dei circuiti, fino alla simulazione di sistemi complessi.

Infine, le **Ising Machines Probabilistiche** rappresentano un passo concreto verso un nuovo paradigma di calcolo ibrido, capace di unire la fisica del rumore termico all'elaborazione computazionale. La ricerca futura si concentrerà sul miglioramento della **scalabilità, adattività e generalizzazione del modello**,

consolidando il ruolo delle architetture SMTJ-pIM come riferimento nella computazione energeticamente efficiente e nell'ottimizzazione di sistemi complessi su larga scala.

Bibliografia

- [1] A. Grimaldi, E. Raimondo, A. Giordano, K. Y. Camsari, and G. Finocchio, "A comparison of energy minimization algorithms for solving Max-SAT problem with probabilistic Ising machines," in *2023 IEEE 23rd International Conference on Nanotechnology (IEEE-NANO)*, pp. 698–702, IEEE, 2023.
- [2] P. J. M. van Laarhoven and E. H. L. Aarts, *Simulated Annealing: Theory and Applications*, Dordrecht: Springer Netherlands, 1987.
- [3] D. Xu and I. Kumazawa, "Single minimum method for combinatorial optimization problems and an efficient algorithm for the TSP problem," in *Proceedings of the 1993 IEEE International Conference on Neural Networks (IJCNN'93)*, pp. 977–982, IEEE, 1993.
- [4] J. Yang, S. Yang, Y. Cen, J. Chen, Y. Huang, Z. Yao, D.-J. Kim, K. Cai, J. Yoo, X. Fong, and H. Yang, "Energy-efficient superparamagnetic Ising machine and its application to traveling salesman problems," *Nature Communications*, vol. 15, no. 1, p. 3457, 2024.
- [5] T. Wang and J. Roychowdhury, "OIM: Oscillator-based Ising machines for solving combinatorial optimisation problems," *arXiv preprint arXiv:1903.07163*, 2019.
- [6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [7] B. Lu, K. Wen, and C. Wang, "Quantum computation using coherent Ising machines based on spiking neural networks," in *2022 14th International Conference on Wireless Communications and Signal Processing (WCSP)*, pp. 813–817, IEEE, 2022.
- [8] N. A. Aadit et al., "A hybrid multi-threaded parallel iterated greedy algorithm for distributed flowshop group scheduling problems with preventive maintenance," in *2024 7th International Conference on Advanced Algorithms and Control Engineering (ICAACE)*, pp. 117–121, IEEE, 2024.
- [9] X. Wang, "Research on solution of TSP based on improved genetic algorithm," in *2018 International Conference on Engineering Simulation and Intelligent Control*, pp. 78–82, IEEE, 2018.
- [10] N. Yoshimura, M. Tawada, S. Tanaka, J. Arai, S. Yagi, H. Uchiyama, and N. Togawa, "Efficient Ising model mapping for induced subgraph isomorphism problems using Ising machines," in *2019 IEEE 9th International Conference on Consumer Electronics (ICCE-Berlin)*, pp. 227–232, IEEE, 2019.
- [11] W. Yang, W. Kong, and Y. Song, "Research on application technology of expert knowledge graph based on greedy iterative search algorithm," in *2024 5th International Conference on Artificial Intelligence and Electromechanical Automation (AIEA)*, pp. 419–422, IEEE, 2024.

- [12] M. A. Mkadem, A. Moukrim, and M. Serairi, "A branch-and-bound algorithm for the two-machine flow-shop problem with time delays," in *Proceedings of the 2017 4th International Conference on Control, Decision and Information Technologies (CoDIT'17)*, pp. 690–695, IEEE, 2017.
- [13] P. Guo, X. Wang, and Y. Han, "A hybrid genetic algorithm for structural optimization with discrete variables," in *2011 International Conference on Internet Computing and Information Services*, pp. 223–226, IEEE, 2011.
- [14] A. Grimaldi, L. Mazza, E. Raimondo, P. Tullo, D. Rodrigues, K. Y. Camsari, V. Crupi, M. Carpentieri, V. Puliafito, and G. Finocchio, "Evaluating spintronics-compatible implementations of Ising machines," *arXiv preprint arXiv:2303.04419*, 2023.
- [15] S. Chowdhury, A. Grimaldi, N. A. Aadit, S. Niazi, M. Mohseni, S. Kanai, H. Ohno, S. Fukami, L. Theogarajan, G. Finocchio, S. Datta, and K. Y. Camsari, "A full-stack view of probabilistic computing with p-bits: Devices, architectures, and algorithms," *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits*, vol. 9, no. 1, pp. 1–11, 2023.
- [16] H. Thimbleby, "Modes, wysiwyg and the von Neumann bottleneck," *International Journal of Man-Machine Studies*, vol. 33, no. 3, pp. 301–323, 1990.
- [17] B. M. McCoy and J.-M. Maillard, "The importance of the Ising model," *Progress of Theoretical Physics*, vol. 127, no. 5, pp. 791–817, 2012.
- [18] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.
- [19] J. J. Hopfield and D. W. Tank, "Neural computation of decisions in optimization problems," *Biological Cybernetics*, vol. 52, pp. 141–152, 1985.
- [20] Y. Nagata and D. Soler, "A new genetic algorithm for the asymmetric traveling salesman problem," *Expert Systems with Applications*, vol. 39, no. 10, pp. 8947–8953, 2012.

Appendice A:

Codice *MATLAB* approccio SHIL Ibrido con Two Opt (N = 12):

```
clear all
close all
clc

cities_data = {
    'Rome',      41.8933, 12.4828;
    'Milan',     45.4669, 9.1900;
    'Naples',    40.8333, 14.2500;
    'Turin',     45.0792, 7.6761;
    'Palermo',   38.1111, 13.3517;
    'Genoa',     44.4111, 8.9328;
    'Bologna',   44.4939, 11.3428;
    'Florence',  43.7714, 11.2542;
    'Bari',      41.1253, 16.8667;
    'Catania',   37.5000, 15.0903;
    'Brescia',   45.5417, 10.2167;
    'Prato',     43.8800, 11.0983;
};

n      = size(cities_data, 1);
cities = cities_data(:,1);
lat    = cell2mat(cities_data(:,2));
lon    = cell2mat(cities_data(:,3));

startIdx = 1;
R        = 10;
A_weight = 1.0;

R_terra_km = 6371.0;
deg2rad = @(d) d*pi/180;
latr = deg2rad(lat);
lonr = deg2rad(lon);

D = zeros(n,n);
for i = 1:n
    for j = 1:n
        dlat = latr(j) - latr(i);
        dlon = lonr(j) - lonr(i);

        a = sin(dlat/2).^2 +
            cos(latr(i)).*cos(latr(j)).*sin(dlon/2).^2;
        c = 2*atan2(sqrt(a), sqrt(1-a));
        D(i,j) = R_terra_km * c;
    end
end
fprintf('Matrice delle distanze Haversine (%dx%d) calcolata.\n', n,
n);
```

```

fprintf('\n--- RISULTATO BASELINE ERMITIANA (R=%d) ---\n', R);
[routeH, costH, spinsH, ttsH] = tsp_shil_fixedstart(D, R,
startIdx);

R_stats = 10;
costsH_stats = zeros(R_stats, 1);
timesH_stats = zeros(R_stats, 1);
fprintf('\n--- ESECUZIONI AGGIUNTIVE PER STATISTICHE ERMITIANE
(R=%d) ---\n', R_stats);

for r_stats = 1:R_stats

    [~, cost_run, ~, tts_run] = tsp_shil_fixedstart(D, 1,
startIdx);

    costsH_stats(r_stats) = cost_run;
    timesH_stats(r_stats) = tts_run;

    fprintf('Ermitiano Stat | Run=%2d | Costo=%.4f km | TTS=%.4f
s\n', r_stats, cost_run, tts_run);
end

meanCostH_stats = mean(costsH_stats);
meanTimeH_stats = mean(timesH_stats);

fprintf('\nRISULTATI BASELINE ERMITIANA (STATISTICA SU %d RUN):\n',
R_stats);
fprintf('1) Costo Ottimo (da riga originale): %.2f km\n', costH);
fprintf('2) Costo Medio (statistico): %.2f km\n', meanCostH_stats);
fprintf('3) TTS Totale (da riga originale): %.3f s\n', ttsH);
fprintf('4) TTS Medio (statistico): %.3f s\n', meanTimeH_stats);

.....

function [bestRoute, bestCost, spinVec, tts] =
tsp_shil_fixedstart(D, R, startIdx)

    if nargin < 2, R = 10; end          % Gestione di R
    if nargin < 3, startIdx = 1; end    % Gestione di startIdx

    tic;

    N    = size(D,1);
    A    = max(D(:)) * N;
    B    = 1;
    nVar = N * N;

    Jp = zeros(nVar); hp = zeros(nVar,1);

    for i = 1:N

```

```

        idx = (i-1)*N + (1:N);
        for j1 = 1:N
            for j2 = j1+1:N
                Jp(idx(j1),idx(j2)) = Jp(idx(j1),idx(j2)) + A/4;
            end
        end
        hp(idx) = hp(idx) + (A/2)*(N-2);
    end
    for j = 1:N
        idx = j:N:nVar;
        for i1 = 1:N
            for i2 = i1+1:N
                Jp(idx(i1),idx(i2)) = Jp(idx(i1),idx(i2)) + A/4;
            end
        end
        hp(idx) = hp(idx) + (A/2)*(N-2);
    end
    for j = 1:N
        jp1 = mod(j,N) + 1;
        for i1 = 1:N
            for i2 = 1:N
                if i1~=i2
                    a = (i1-1)*N + j;
                    b = (i2-1)*N + jp1;
                    Jp(a,b) = Jp(a,b) + B/4 * D(i1,i2);
                    hp(a) = hp(a) + B/4 * D(i1,i2);
                    hp(b) = hp(b) + B/4 * D(i1,i2);
                end
            end
        end
    end
end

J = -(Jp + Jp. ');
h = -hp;

omega0 = 1;
epsilon = 0.01;
Kc = epsilon/omega0;
Ks = epsilon;

sigma0 = 0.05;
sigmaMin = 0.001;
dt = 0.001;
Tmax = 500;

bestCost = inf;
bestRoute = [];

for run = 1:R
    rng('shuffle');
    phi = 2*pi*rand(nVar,1);
    sigma = sigma0;

```

```

ds    = (sigma0 - sigmaMin)/Tmax;

for t = 1:Tmax
    Delta = phi - phi.';
    coup  = sum(J .* sin(Delta),2);
    shil  = -Ks * sin(2*phi);
    dphi  = Kc * coup + shil;
    phi   = phi + dt*dphi +
sqrt(dt)*sigma*randn(size(phi));
    sigma = max(sigma - ds, sigmaMin);
end

phi_mat = reshape(phi,[N,N]);
score   = cos(phi_mat);
assign  = zeros(1,N);
used    = false(1,N);
for j = 1:N
    [~, ids] = sort(score(:,j),'descend');
    for k = ids'
        if ~used(k)
            assign(j) = k;
            used(k)   = true;
            break;
        end
    end
end
u = unique(assign);
if numel(u) < N
    miss = setdiff(1:N, u);
    counts = histcounts(assign, 1:N+1);
    dups   = find(counts > 1);
    for j = 1:N
        if ismember(assign(j), dups)
            assign(j) = miss(1);
            miss(1)   = [];
            counts    = histcounts(assign, 1:N+1);
            dups      = find(counts > 1);
            if isempty(miss), break; end
        end
    end
end
route0 = [assign assign(1)];
[route, cost] = two_opt(route0, D);

if cost < bestCost
    bestCost = cost;
    bestRoute = route;
end
end

```

```

    k = find(bestRoute == startIdx, 1);
    bestRoute = circshift(bestRoute, [0, -(k-1)]);
    bestRoute(end) = startIdx;

    spinVec = zeros(N*N,1);
    for k = 1:(numel(bestRoute)-1)
        i = bestRoute(k);
        j = bestRoute(k+1);
        spinVec((i-1)*N + j) = -1;
        spinVec((j-1)*N + i) = +1;
    end

    tts = toc;
end

function [route, cost] = two_opt(route0, D)
    route = route0;
    cost = tour_cost(route,D);
    improved = true;
    while improved
        improved = false;
        Np1 = numel(route);
        for i = 2:Np1-2
            for j = i+1:Np1-1
                newr = [route(1:i-1), route(j:-1:i), route(j+1:end)];
                nc = tour_cost(newr,D);
                if nc < cost
                    route = newr;
                    cost = nc;
                    improved = true;
                    break;
                end
            end
        end
        if improved, break; end
    end
end

function c = tour_cost(route, D)
    c = 0;
    for k = 1:numel(route)-1
        c = c + D(route(k),route(k+1));
    end
end

```

Appendice B:

Codice *MATLAB* approccio SHIL senza *Two Opt* (N = 12):

```
clear all
close all
clc

cities = {
    'Rome',      41.8933, 12.4828;
    'Milan',     45.4669, 9.1900;
    'Naples',    40.8333, 14.2500;
    'Turin',     45.0792, 7.6761;
    'Palermo',   38.1111, 13.3517;
    'Genoa',     44.4111, 8.9328;
    'Bologna',   44.4939, 11.3428;
    'Florence',  43.7714, 11.2542;
    'Bari',      41.1253, 16.8667;
    'Catania',   37.5000, 15.0903;
    'Brescia',   45.5417, 10.2167;
    'Prato',     43.8800, 11.0983;
};

n = size(cities,1);
names = cities(:,1);
lat  = cell2mat(cities(:,2));
lon  = cell2mat(cities(:,3));

startIdx = 1;
R         = 10;

R_terra_km = 6371.0;
deg2rad = @(d) d*pi/180;
latr = deg2rad(lat);
lonr = deg2rad(lon);

D = zeros(n,n);
for i = 1:n
    for j = 1:n
        dlat = latr(j) - latr(i);
        dlon = lonr(j) - lonr(i);
        a = sin(dlat/2).^2 +
            cos(latr(i)).*cos(latr(j)).*sin(dlon/2).^2;
        c = 2*atan2(sqrt(a), sqrt(1-a));
        D(i,j) = R_terra_km * c;
    end
end

fprintf('Matrice delle distanze Haversine (6x6) calcolata.\n');

R_stats = R;
costs_pure = zeros(R_stats, 1);
times_pure = zeros(R_stats, 1);
```

```

cost_best = inf;
route_best = [];
tts_best = 0;

fprintf('\n--- ESECUZIONI STATISTICHE PURE SHIL (R=%d) ---\n',
R_stats);

for r_stats = 1:R_stats
    [route_run, cost_run, ~, tts_run] = tsp_shil_pure_ising(D, 1,
startIdx);

    costs_pure(r_stats) = cost_run;
    times_pure(r_stats) = tts_run;

    if cost_run < cost_best
        cost_best = cost_run;
        route_best = route_run;
        tts_best = tts_run;
    end

    fprintf('Pure SHIL Stat | Run=%2d | Costo=%.4f km | TTS=%.4f
s\n', r_stats, cost_run, tts_run);
end

meanCost_pure = mean(costs_pure);
meanTime_pure = mean(times_pure);
stdCost_pure = std(costs_pure);

fprintf('\n=====
');
fprintf('RISULTATI PURE SHIL MACHINE (STATISTICA SU %d RUN):\n',
R_stats);
fprintf('=====
');
;
fprintf('1) Costo Assoluto Migliore: %.2f km\n', cost_best);
fprintf('2) Costo Medio Statistico: %.2f km (Dev. Std: %.2f)\n',
meanCost_pure, stdCost_pure);
fprintf('3) TTS Assoluto Migliore: %.3f s\n', tts_best);
fprintf('4) TTS Medio Statistico: %.3f s\n', meanTime_pure);

.....

function [bestRoute, bestCost, spinVec, tts] =
tsp_shil_pure_ising(D, R, startIdx)

    if nargin < 2, R = 10; end
    if nargin < 3, startIdx = 1; end

    tic;
    N = size(D,1);
    A = max(D(:)) * N;
    B = 1;

```



```

nVar = N * N;

Jp = zeros(nVar);
hp = zeros(nVar,1);

for i = 1:N
    idx = (i-1)*N + (1:N);
    for j1 = 1:N
        for j2 = j1+1:N
            Jp(idx(j1),idx(j2)) = Jp(idx(j1),idx(j2)) + A/4;
        end
    end
    hp(idx) = hp(idx) + (A/2)*(N-2);
end
for j = 1:N
    idx = j:N:nVar;
    for i1 = 1:N
        for i2 = i1+1:N
            Jp(idx(i1),idx(i2)) = Jp(idx(i1),idx(i2)) + A/4;
        end
    end
    hp(idx) = hp(idx) + (A/2)*(N-2);
end
for j = 1:N
    jp1 = mod(j,N) + 1;
    for i1 = 1:N
        for i2 = 1:N
            if i1~=i2
                a = (i1-1)*N + j;
                b = (i2-1)*N + jp1;
                Jp(a,b) = Jp(a,b) + B/4 * D(i1,i2);
                hp(a) = hp(a) + B/4 * D(i1,i2);
                hp(b) = hp(b) + B/4 * D(i1,i2);
            end
        end
    end
end
J = -(Jp + Jp. ');
h = -hp;

omega0 = 1;
epsilon = 0.01;
Kc = epsilon/omega0;
Ks = epsilon;

sigma0 = 0.1;
sigmaMin = 0.001;
dt = 0.001;
Tmax = 10000;

bestCost = inf;
bestRoute = [];

```

```

for run = 1:R
    rng('shuffle');
    phi = 2*pi*rand(nVar,1);
    sigma = sigma0;
    ds = (sigma0 - sigmaMin)/Tmax;

    for t = 1:Tmax
        Delta = phi - phi.';
        coup = sum(J .* sin(Delta),2);
        shil = -Ks * sin(2*phi);
        dphi = Kc * coup + shil;
        phi = phi + dt*dphi +
sqrt(dt)*sigma*randn(size(phi));
        sigma = max(sigma - ds, sigmaMin);
    end

    phi_mat = reshape(phi,[N,N]);
    score = cos(phi_mat);
    assign = zeros(1,N);
    used = false(1,N);
    for j = 1:N
        [~, ids] = sort(score(:,j),'descend');
        for k = ids'
            if ~used(k)
                assign(j) = k;
                used(k) = true;
                break;
            end
        end
    end
end

u = unique(assign);
if numel(u) < N
    miss = setdiff(1:N, u);
    counts = histcounts(assign, 1:N+1);
    dups = find(counts > 1);
    for j = 1:N
        if ismember(assign(j), dups)
            assign(j) = miss(1);
            miss(1) = [];
            counts = histcounts(assign, 1:N+1);
            dups = find(counts > 1);
            if isempty(miss), break; end
        end
    end
end

route = [assign assign(1)];
cost = tour_cost(route, D);
if cost < bestCost
    bestCost = cost;
end

```

```

        bestRoute = route;
    end
end

k = find(bestRoute == startIdx, 1);
bestRoute = circshift(bestRoute, [0, -(k-1)]);
bestRoute(end) = startIdx;

spinVec = zeros(N*N,1);
for k = 1:(numel(bestRoute)-1)
    i = bestRoute(k);
    j = bestRoute(k+1);

    spinVec((i-1)*N + j) = -1;
    spinVec((j-1)*N + i) = +1;
end

tts = toc;
end

function c = tour_cost(route, D)
    c = 0;
    for k = 1:numel(route)-1
        c = c + D(route(k),route(k+1));
    end
end
end

```

Appendice C:

Codice *MATLAB* approccio *Brute Force* (N=12):

```
clear all;
close all;
clc;

cities = {
    'Rome',      41.8933, 12.4828;
    'Milan',     45.4669, 9.1900;
    'Naples',    40.8333, 14.2500;
    'Turin',     45.0792, 7.6761;
    'Palermo',   38.1111, 13.3517;
    'Genoa',     44.4111, 8.9328;
    'Bologna',   44.4939, 11.3428;
    'Florence',  43.7714, 11.2542;
    'Bari',      41.1253, 16.8667;
    'Catania',   37.5000, 15.0903;
    'Brescia',   45.5417, 10.2167;
    'Prato',     43.8800, 11.0983;
};

n = size(cities,1);
names = cities(:,1);
lat = cell2mat(cities(:,2));
lon = cell2mat(cities(:,3));

startIdx = 1; % Bari
R = 10; % Numero di run per la media statistica

% -----
% Matrice distanze Haversine (km)
% -----
R_terra_km = 6371.0;
deg2rad = @(d) d*pi/180;
latr = deg2rad(lat);
lonr = deg2rad(lon);

D = zeros(n,n);
for i = 1:n
    for j = 1:n
        dlat = latr(j) - latr(i);
        dlon = lonr(j) - lonr(i);
        % Formula di Haversine
        a = sin(dlat/2).^2 +
cos(latr(i)).*cos(latr(j)).*sin(dlon/2).^2;
        c = 2*atan2(sqrt(a), sqrt(1-a));
        D(i,j) = R_terra_km * c;
    end
end
end
```

```

fprintf('Matrice delle distanze Haversine (6x6) calcolata.\n');

% -----
% Solver PURE SHIL (Senza Two-Opt)
% -----

R_stats = R; % Usa R run per le statistiche
costs_pure = zeros(R_stats, 1);
times_pure = zeros(R_stats, 1);
cost_best = inf;
route_best = [];
tts_best = 0;

fprintf('\n--- ESECUZIONI STATISTICHE PURE SHIL (R=%d) ---\n',
R_stats);

for r_stats = 1:R_stats
    % Esegui il solver Pure SHIL con R=1 (una singola run alla
    volta)
    % Assicurati che 'tsp_shil_pure_ising.m' sia nel path di
    MATLAB.
    [route_run, cost_run, ~, tts_run] = tsp_shil_pure_ising(D, 1,
startIdx);

    costs_pure(r_stats) = cost_run;
    times_pure(r_stats) = tts_run;

    % Aggiorna il percorso migliore
    if cost_run < cost_best
        cost_best = cost_run;
        route_best = route_run;
        tts_best = tts_run;
    end

    % Visualizza i dati della singola run
    fprintf('Pure SHIL Stat | Run=%2d | Costo=%.4f km | TTS=%.4f
s\n', r_stats, cost_run, tts_run);
end

% -----
% Calcolo Statistiche e Output
% -----

meanCost_pure = mean(costs_pure);
meanTime_pure = mean(times_pure);
stdCost_pure = std(costs_pure);

fprintf('\n=====
');
fprintf('RISULTATI PURE SHIL MACHINE (STATISTICA SU %d RUN):\n',
R_stats);

```

```

fprintf('=====\n')
;
fprintf('1) Costo Assoluto Migliore: %.2f km\n', cost_best);
fprintf('2) Costo Medio Statistico: %.2f km (Dev. Std: %.2f)\n',
meanCost_pure, stdCost_pure);
fprintf('3) TTS Assoluto Migliore: %.3f s\n', tts_best);
fprintf('4) TTS Medio Statistico: %.3f s\n', meanTime_pure);
fprintf('-----\n');

% --- Visualizza tour migliore ---
fprintf('Ordine di visita del tour Migliore (Pure SHIL): ');
for i = 1:numel(route_best)
    fprintf('%s', names{route_best(i)});
    if i < numel(route_best), fprintf(' -> '); end
end
fprintf('\n');

% --- Confronto con l'ottimo esatto (tsp_italia.m) ---
% Eseguiamo il solver esatto (Brute Force) per confronto
others = 2:n;
bestLen_exact = inf;
bestTour_exact = [];
tic;
P = perms(others);
% CORREZIONE DELL'ERRORE: La sintassi del loop "for" è pulita qui.
for k = 1:size(P,1)
    tour = [startIdx, P(k,:), startIdx];
    len = 0;
    for t = 1:numel(tour)-1
        len = len + D(tour(t), tour(t+1));
    end
    if len < bestLen_exact
        bestLen_exact = len;
        bestTour_exact = tour;
    end
end
elapsedTime_exact = toc;

fprintf('\n=====\n');
fprintf('RISULTATI BRUTE FORCE (Benchmark):\n');
fprintf('=====\n');
;
fprintf('Distanza minima esatta: %.2f km\n', bestLen_exact);
fprintf('Itinerario ottimo esatto: ');
for i = 1:numel(bestTour_exact)
    fprintf('%s', names{bestTour_exact(i)});
    if i < numel(bestTour_exact), fprintf(' -> '); end
end
fprintf('\n');

```

```
fprintf('Time-to-solution esatto: %.6f s (permutazioni=%d)\n',  
elapsedTime_exact, size(P,1));  
fprintf('-----  
\n');
```